

本节内容

散列表的 基本概念

知识总览

散列表的基本概念

散列表、散列函数

冲突、同义词

关于散列表，有待解决的问题？

散列表、散列函数

散列表（哈希表，Hash Table）：是一种数据结构。特点是：可以根据数据元素的关键字计算出它在散列表中的存储地址

散列函数（哈希函数）： $\text{Addr}=\text{H}(\text{key})$ 建立了“关键字”→“存储地址”的映射关系。

例：某散列表的长度为13，散列函数 $\text{H}(\text{key})=\text{key}\%13$ 。依次将数据元素 19、14、23 插入散列表：

$$19\%13=6$$

$$14\%13=1$$

$$23\%13=10$$

	14					19				23		
0	1	2	3	4	5	6	7	8	9	10	11	12

理想情况下，在散列表中查找一个元素的时间复杂度为 $O(1)$ 。比如：

查找元素 19：根据散列函数计算出元素的存储地址= $19\%13=6$ ，检查位置#6。查找成功

查找元素 16：根据散列函数计算出元素的存储地址= $16\%13=3$ ，检查位置#3。查找失败

冲突、同义词

冲突（碰撞）：在散列表中插入一个数据元素时，需要根据关键字的值确定其存储地址，若该地址已经存储了其他元素，则称这种情况为“冲突（碰撞）”

同义词：若不同的关键字通过散列函数映射到同一个存储地址，则称它们为“同义词”

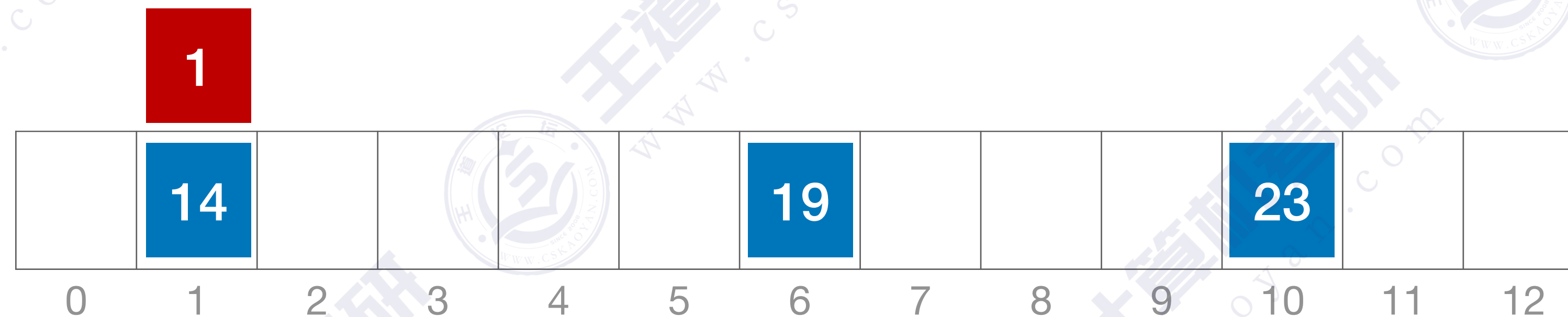
例：某散列表的长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ 。依次将数据元素 19、14、23 插入散列表：

$$19 \% 13 = 6$$

$$14 \% 13 = 1$$

$$23 \% 13 = 10$$

$$1 \% 13 = 1$$

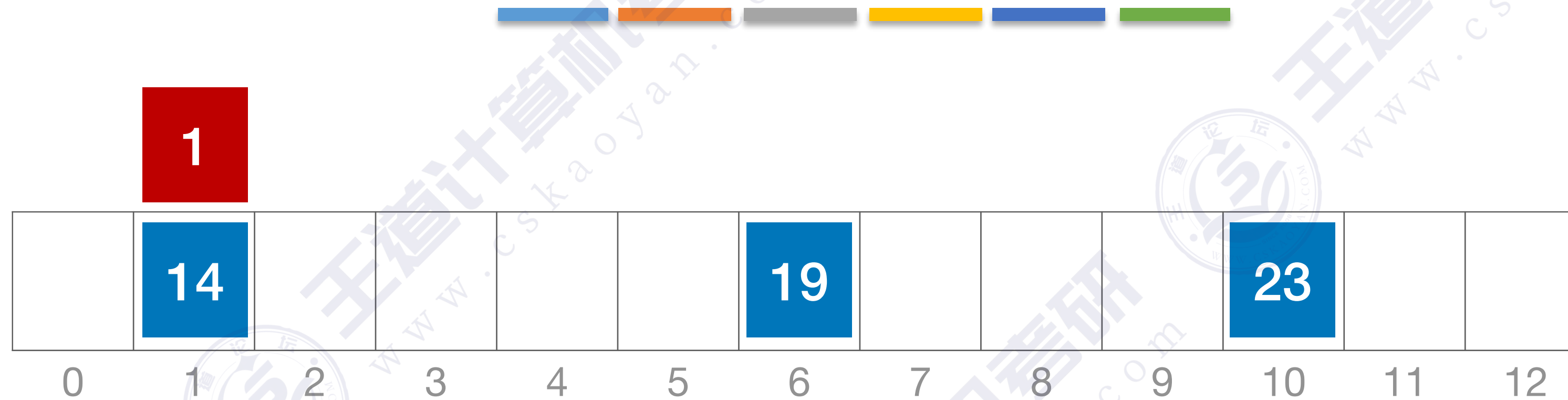


继续插入元素 1：存储地址 $= 1 \% 13 = 1$ ，该地址已经存储了其他元素——发生“冲突”

对于散列函数 $H(\text{key}) = \text{key} \% 13$ 来说，1 和 14 是“同义词”



如何减少“冲突”?



对于散列函数 $H(\text{key}) = \text{key} \% 13$ 来说, 1 和 14 是“同义词”

如何减少“冲突”?

构造更适合的散列函数, 让各个关键字尽可能地映射到不同的存储位置, 从而减少“冲突”



Eg: 把散列函数改为 $H(\text{key}) = \text{key} \% 12$, 则不发生冲突

$19 \% 12 = 7$
 $14 \% 12 = 2$
 $23 \% 12 = 11$
 $1 \% 12 = 1$

待解决的问题: 如何构造散列函数?

若“冲突”无可避免，如何处理冲突？



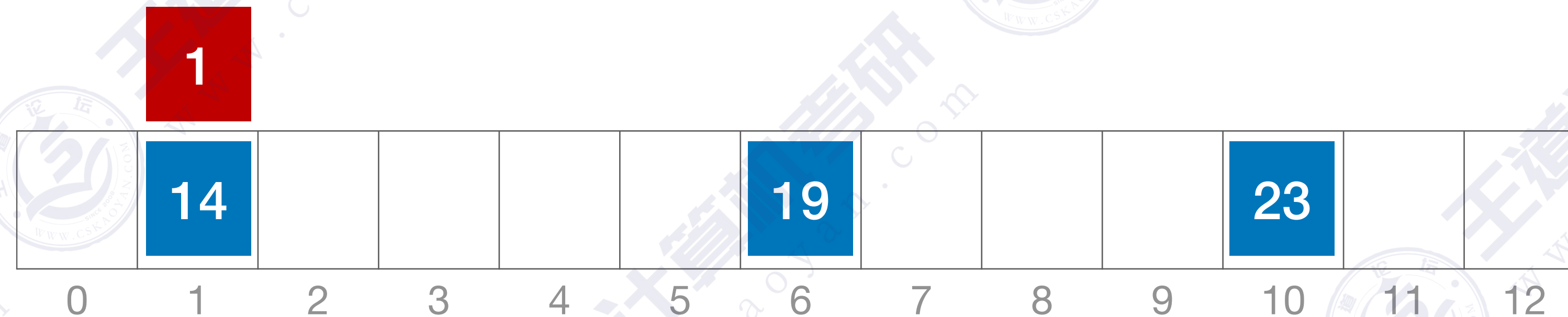
例：某散列表的长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ 。依次将数据元素 19、14、23、1 插入散列表：

$$19 \% 13 = 6$$

$$14 \% 13 = 1$$

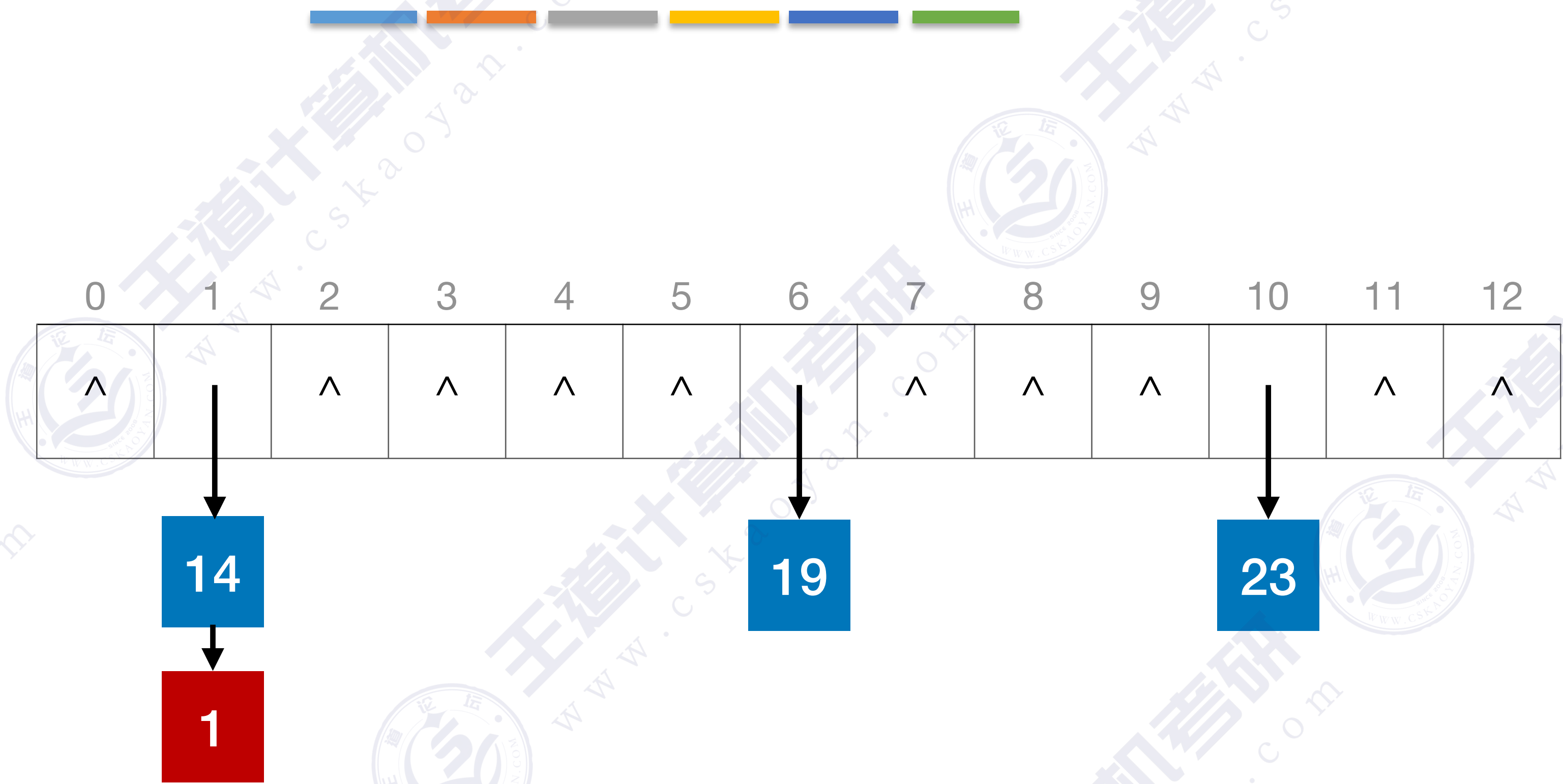
$$23 \% 13 = 10$$

$$1 \% 13 = 1$$



如何处理冲突？——拉链法

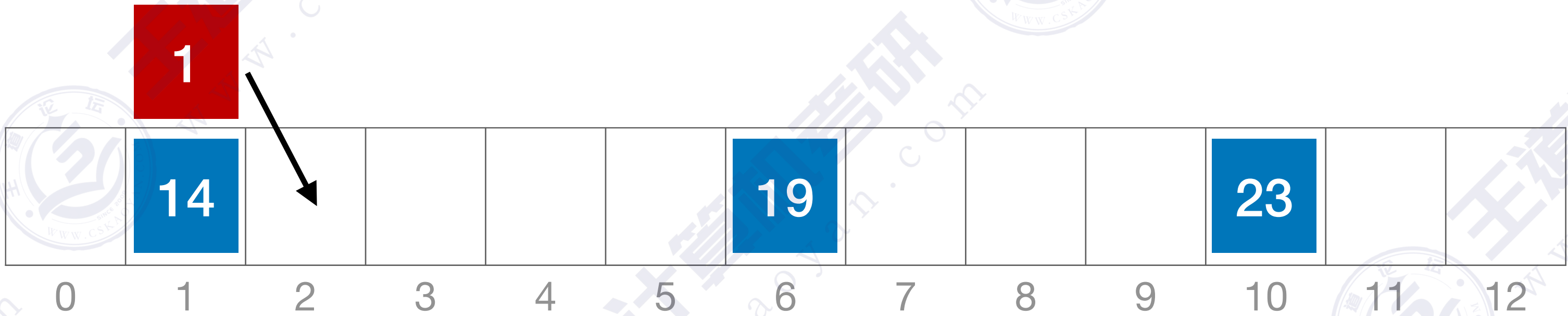
19%13=6
14%13=1
23%13=10
1%13=1



拉链法（又称链接法、链地址法）：把所有“同义词”存储在一个链表中

如何处理冲突？——开放定址法

$19\%13=6$
 $14\%13=1$
 $23\%13=10$
 $1\%13=1$



开放定址法：如果发生“冲突”，就给新元素找另一个空闲位置。

待解决的问题：用什么规则确定“另一个空闲位置”？

知识回顾与重要考点

散列表的基本概念

- 散列表 —— 又称 哈希表，可以根据数据元素的关键字计算出它在散列表中的存储地址
- 散列函数 —— 建立了“关键字”→“存储地址”的映射关系。
- 同义词 —— 若不同的关键字通过散列函数映射到同一个存储地址，则称它们为“同义词”
- 冲突（碰撞） —— 在散列表中插入一个数据元素时，若插入的位置已经存储了其他元素，则称这种情况为“冲突（碰撞）”
- 有待解决的问题
 - 如何减少“冲突”？ —— 构造更适合的散列函数
 - 如何处理“冲突”？
 - 拉链法
 - 开放定址法

本节内容

散列函数的构造

知识总览

核心目标：散列函数要尽可能地减少“冲突”

散列函数的构造

设计散列函数时应该注意什么？

★ 除留余数法

直接定址法

数字分析法

平方取中法

适用不同场景

设计散列函数时应该注意什么？

散列函数（哈希函数）： $\text{Addr} = H(\text{key})$ 建立了“关键字”→“存储地址”的映射关系。

设计散列函数时应该注意什么：

1. 定义域必须涵盖所有可能出现的关键字。

反例： $H(\text{key}) = \sqrt{\text{key}} \% 13$ ，不支持关键字为负值

2. 值域不能超出散列表的地址范围。

反例： $H(\text{key}) = \text{key} \% 15$ ，可能被映射到非法地址13、14

3. 尽可能减少冲突。散列函数计算出来的地址应尽可能均匀分布在整个地址空间。

反例： $H(\text{key}) = (\text{key} \times 13) \% 13$
任何关键字都会被映射到地址0

4. 散列函数应尽量简单，能够快速计算出任意一个关键字对应的散列地址。

反例： $H(\text{key}) = (\text{key}!) \% 13$ ，若关键字值较大，计算阶乘耗时高

例：散列表长度为13，关键字可能是任一“整数”

0	1	2	3	4	5	6	7	8	9	10	11	12

除留余数法

除留余数法 —— $H(key) = key \% p$

散列表表长为 m ，取一个不大于 m 但最接近或等于 m 的质数 p

注：质数又称素数。指除了1和此整数自身外,不能被其他自然数整除的数

适用场景：较为通用，只要关键字是整数即可

例：散列表表长 $m=13$ 。13是质数，因此可令散列函数 $H(key)=key\%13$

0	1	2	3	4	5	6	7	8	9	10	11	12

例：散列表表长 $m=15$ 。不大于15且最接近15的质数是13，因此可令散列函数 $H(key)=key\%13$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
													^	^

拓展：为什么除留余数法要对质数取余？

除留余数法 —— $H(key) = key \% p$

散列表表长为m，取一个不大于m但最接近或等于m的质数p

注：质数又称素数。指除了1和此整数自身外,不能被其他自然数整除的数

原因：对质数取余，可以分布更均匀，从而减少冲突

设：可能出现的关键字={2,4,6,8,10,12,14}

散列表表长8，散列函数 $H(key)=key\%8$

0	1	2	3	4	5	6	7

8	2	4	6
	10	12	14

对合数取余，散列地址分布不均匀，易发生冲突

散列表表长8，散列函数 $H(key)=key\%7$

0	1	2	3	4	5	6	7
							^

14	8	2	10	4	12	6
----	---	---	----	---	----	---

对质数取余，散列地址分布均匀，不易发生冲突

Why? ——取余运算会被“公因子”影响，参见《数论》

直接定址法

直接定址法 —— $H(\text{key}) = \text{key}$ 或 $H(\text{key}) = a * \text{key} + b$

其中，a和b是常数。这种方法计算最简单，且不会产生冲突。若关键字分布不连续，空位较多，则会造成存储空间的浪费。

适用场景：关键字分布基本连续

例：用散列表存储某班级的30个学生信息，班内学生学号为(1120112176~1120112205)

可令 $H(\text{key}) = \text{key} - 1120112176$

0	1	2	3	4	5	...	...	...	26	27	28	29
...176	...177	...178	...179	...180	...181	...	...	...	...202	...203	...204	...205

数字分析法

数字分析法 —— 选取数码分布较为均匀的若干位作为散列地址

设关键字是 r 进制数（如十进制数），而 r 个数码在各位上出现的频率不一定相同，可能在某些位上分布均匀一些，每种数码出现的机会均等；而在某些位上分布不均匀，只有某几种数码经常出现，此时可**选取数码分布较为均匀的若干位作为散列地址**。

适用场景：关键字集合已知，且关键字的某几个数码位分布均匀

例：要求将手机用户的信息存入长度为 10000 的散列表。以“手机号码”作为关键字设计散列函数

138XXXX2875
138XXXX1682
138XXXX9125
.....
199XXXX1684
199XXXX1236

以手机号后四位作为散列地址

0	1	2	3	...	...	9998	9999

平方取中法

平方取中法——取关键字的平方值的中间几位作为散列地址。

具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀。

适用场景：关键字的每位取值都不够均匀。

例：某汽车品牌，以（底盘号、发动机号、外观号）作为一款汽车型号的标识。该品牌有100种车型，欲将这这些车型信息存入长度为100的散列表，如何设计散列函数？

底盘号有：28、34、39、60...

发动机号有：50、52、54...

外观号有：20、25、30、35...

车型号	平方	取中
605030	→ 366061300900	13
345220	→ 119176848400	84
395225	→ 156202800625	28
605435	→ 366551539225	15

985
×985
4925
7880
8865
970225

关键字的各个数码位都分布不均匀

中间几位的值受每个数码位影响

知识回顾与重要考点

散列函数的构造

★ 除留余数法

散列地址： $H(\text{key}) = \text{key} \% p$ 。其中， p 通常是不大于散列表表长的最大质数

适用场景：较为通用，只要关键字是整数即可

直接定址法

散列地址： $H(\text{key}) = a * \text{key} + b$ 。其中， a 和 b 是常数。

适用场景：关键字分布基本连续

数字分析法

选取数码分布较为均匀的若干位作为散列地址

适用场景：关键字集合已知，且关键字的某几个数码位分布均匀

平方取中法

计算关键字的平方，取中间几位作为散列地址

适用场景：关键字的每位取值都不够均匀

本节内容

处理冲突的 方法

——拉链法

知识总览

处理冲突的方法 ——拉链法

如何插入一个元素

如何查找一个元素

—— 学会分析查找长度

如何删除一个元素

Tips: 散列表通常不考代码，着重掌握手算分析方法

散列表的插入操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

0	1	2	3	4	5	6	7	8	9	10	11	12
^	^	^	^	^	^	^	^	^	^	^	^	^

可以视为
13个链表的
头指针

$19 \% 13 = 6$	$27 \% 13 = 1$
$14 \% 13 = 1$	$55 \% 13 = 3$
$23 \% 13 = 10$	$11 \% 13 = 11$
$1 \% 13 = 1$	$10 \% 13 = 10$
$68 \% 13 = 3$	$79 \% 13 = 1$
$20 \% 13 = 7$	
$84 \% 13 = 6$	

拉链法（又称链接法、链地址法）：把所有“同义词”存储在一个链表中

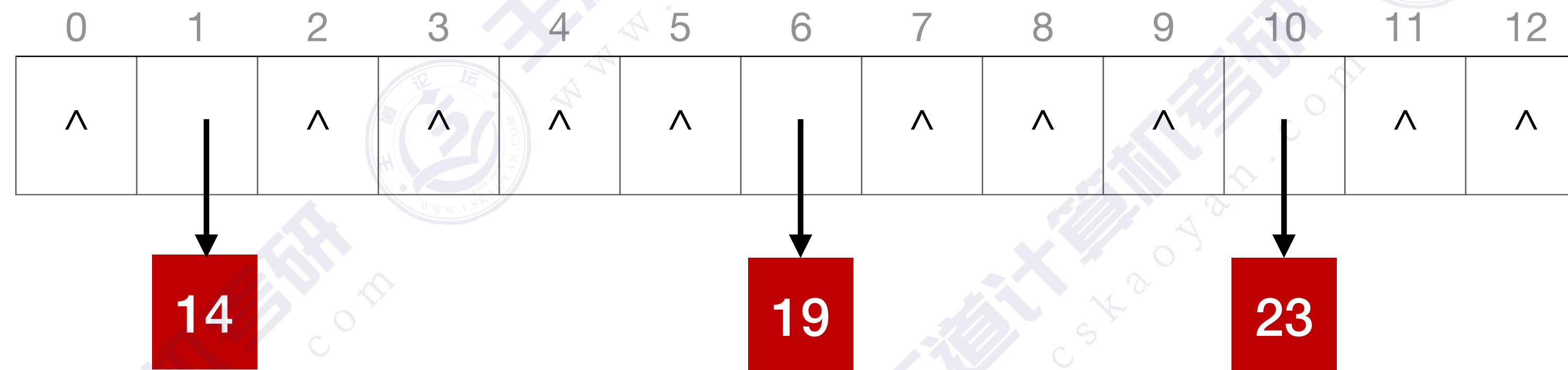
如何在散列表（拉链法解决冲突）中插入一个新元素？

Step 1: 结合散列函数计算新元素的散列地址

Step 2: 将新元素插入散列地址对应的链表（可用头插法，也可用尾插法）

散列表的插入操作（拉链法解决冲突）

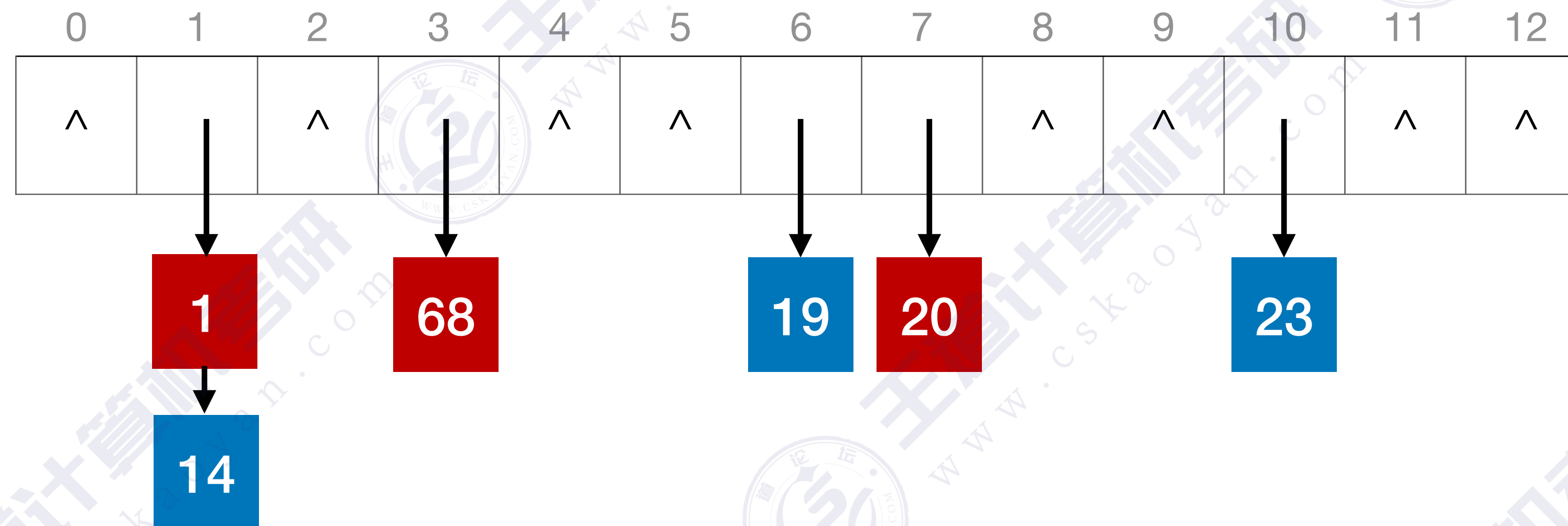
例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}



$19 \% 13 = 6$	$27 \% 13 = 1$
$14 \% 13 = 1$	$55 \% 13 = 3$
$23 \% 13 = 10$	$11 \% 13 = 11$
$1 \% 13 = 1$	$10 \% 13 = 10$
$68 \% 13 = 3$	$79 \% 13 = 1$
$20 \% 13 = 7$	
$84 \% 13 = 6$	

散列表的插入操作（拉链法解决冲突）

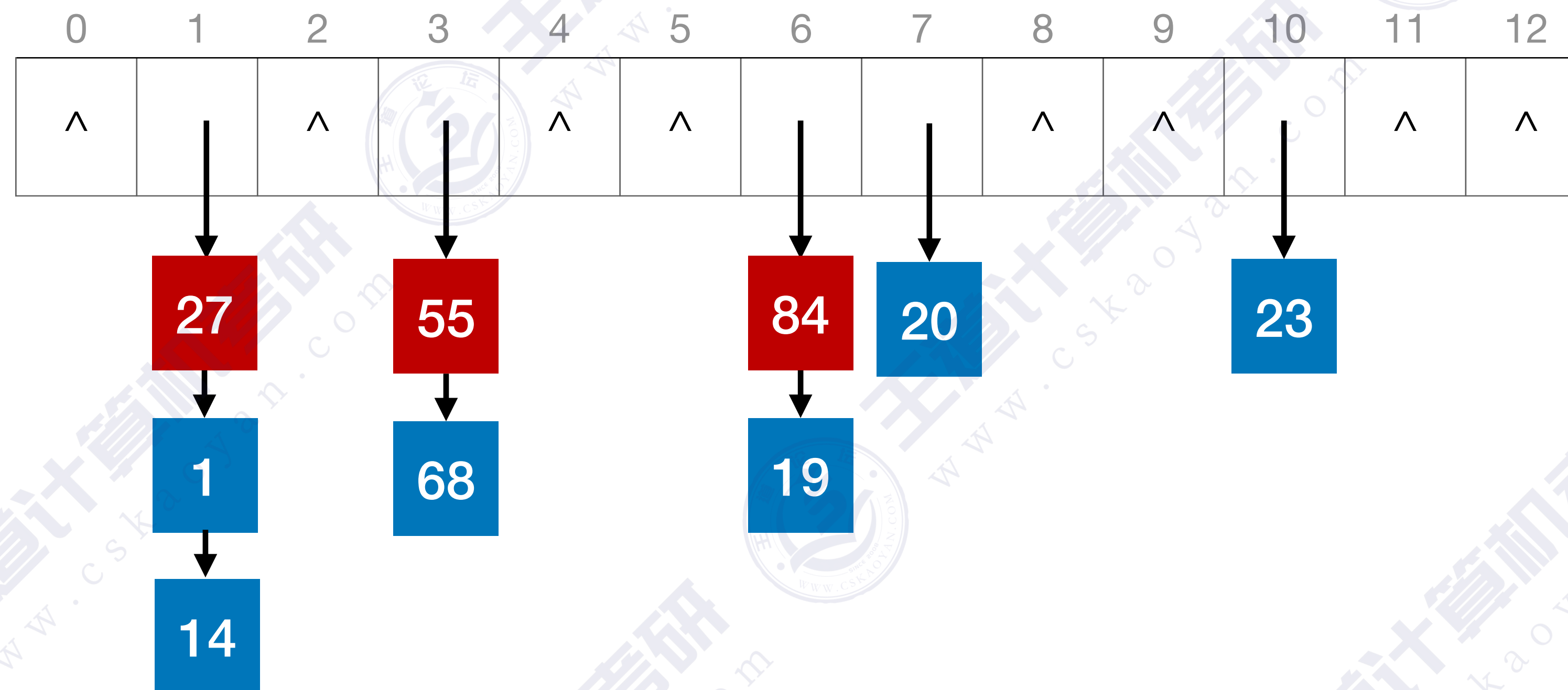
例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}



$19 \% 13 = 6$	$27 \% 13 = 1$
$14 \% 13 = 1$	$55 \% 13 = 3$
$23 \% 13 = 10$	$11 \% 13 = 11$
$1 \% 13 = 1$	$10 \% 13 = 10$
$68 \% 13 = 3$	$79 \% 13 = 1$
$20 \% 13 = 7$	
$84 \% 13 = 6$	

散列表的插入操作（拉链法解决冲突）

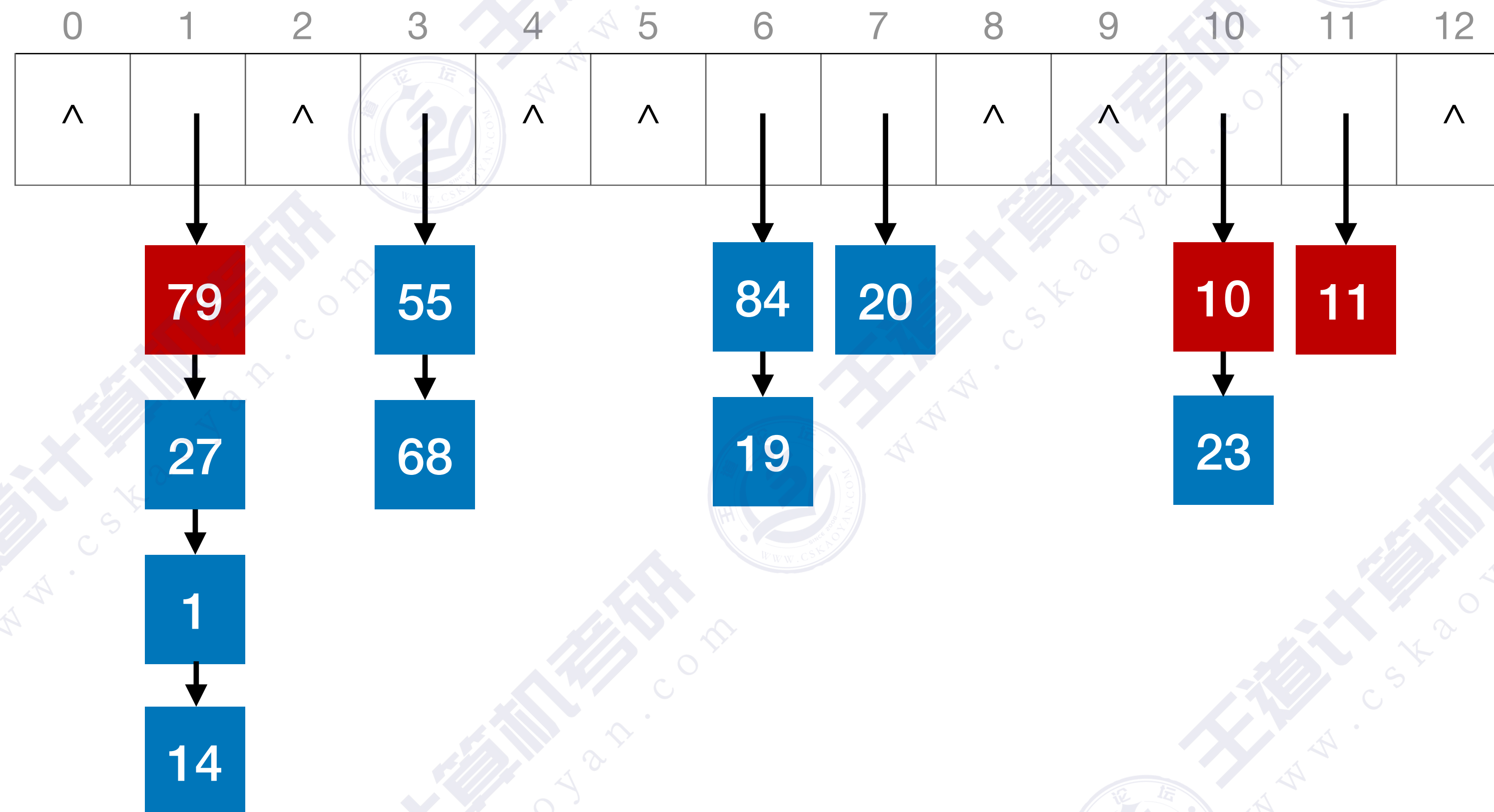
例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}



$19 \% 13 = 6$	$27 \% 13 = 1$
$14 \% 13 = 1$	$55 \% 13 = 3$
$23 \% 13 = 10$	$11 \% 13 = 11$
$1 \% 13 = 1$	$10 \% 13 = 10$
$68 \% 13 = 3$	$79 \% 13 = 1$
$20 \% 13 = 7$	
$84 \% 13 = 6$	

散列表的插入操作（拉链法解决冲突）

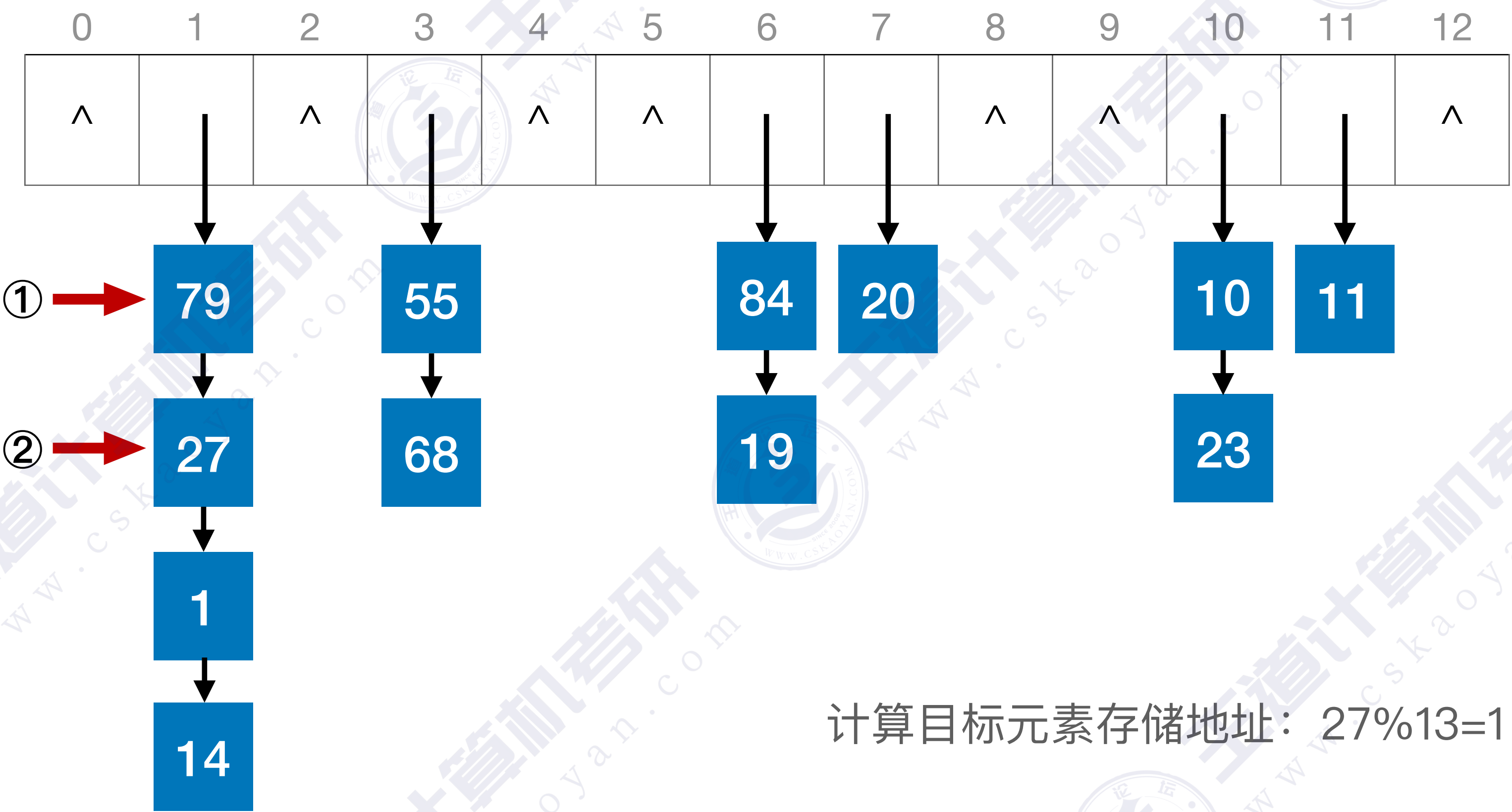
例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}



$19 \% 13 = 6$	$27 \% 13 = 1$
$14 \% 13 = 1$	$55 \% 13 = 3$
$23 \% 13 = 10$	$11 \% 13 = 11$
$1 \% 13 = 1$	$10 \% 13 = 10$
$68 \% 13 = 3$	$79 \% 13 = 1$
$20 \% 13 = 7$	
$84 \% 13 = 6$	

散列表的查找操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27，20，66，21，分析每次查找操作的过程和查找长度。



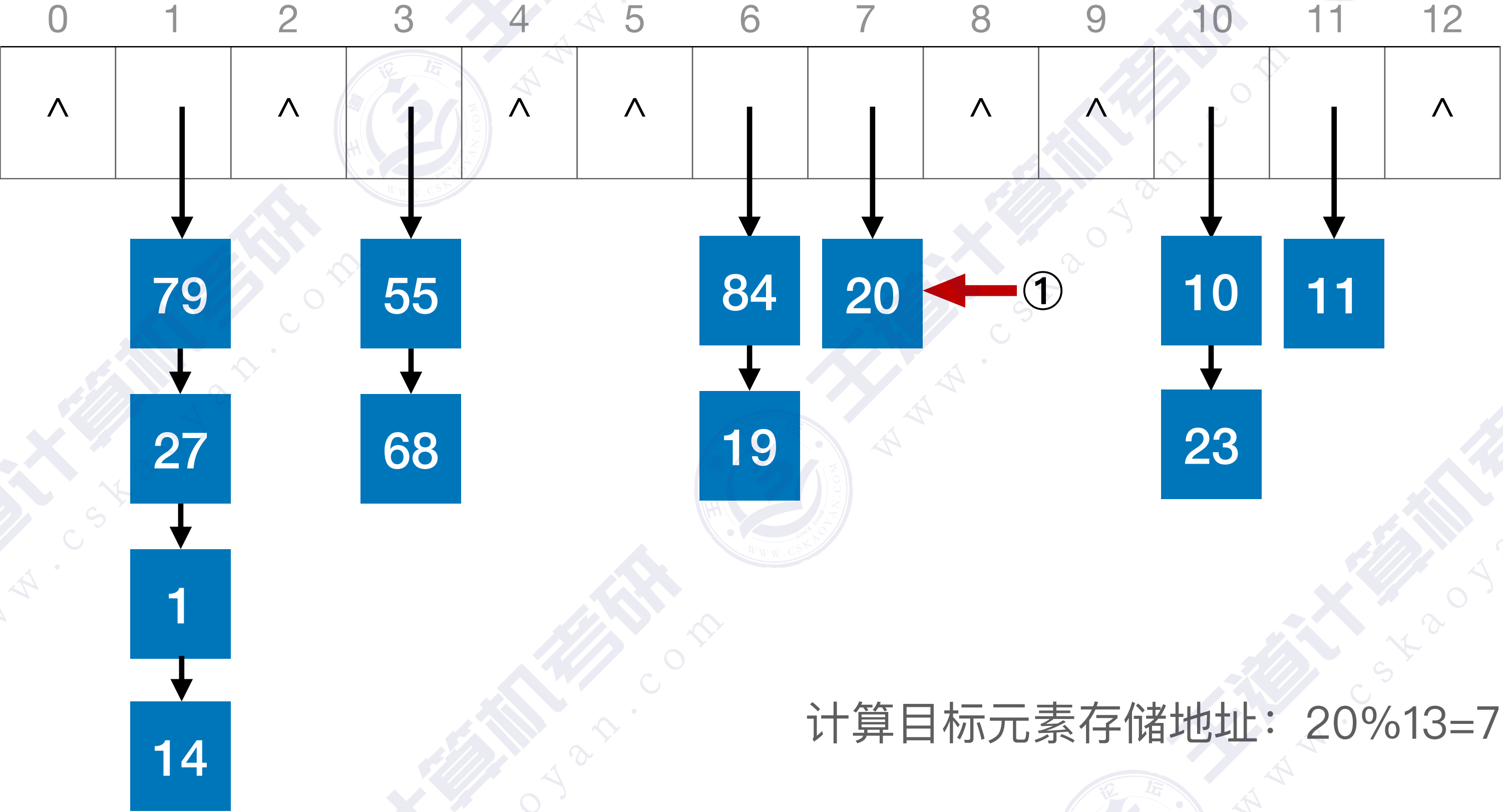
查找目标：27

计算目标元素存储地址： $27\%13=1$

27查找成功，查找长度=2

散列表的查找操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27, 20, 66, 21，分析每次查找操作的过程和查找长度。



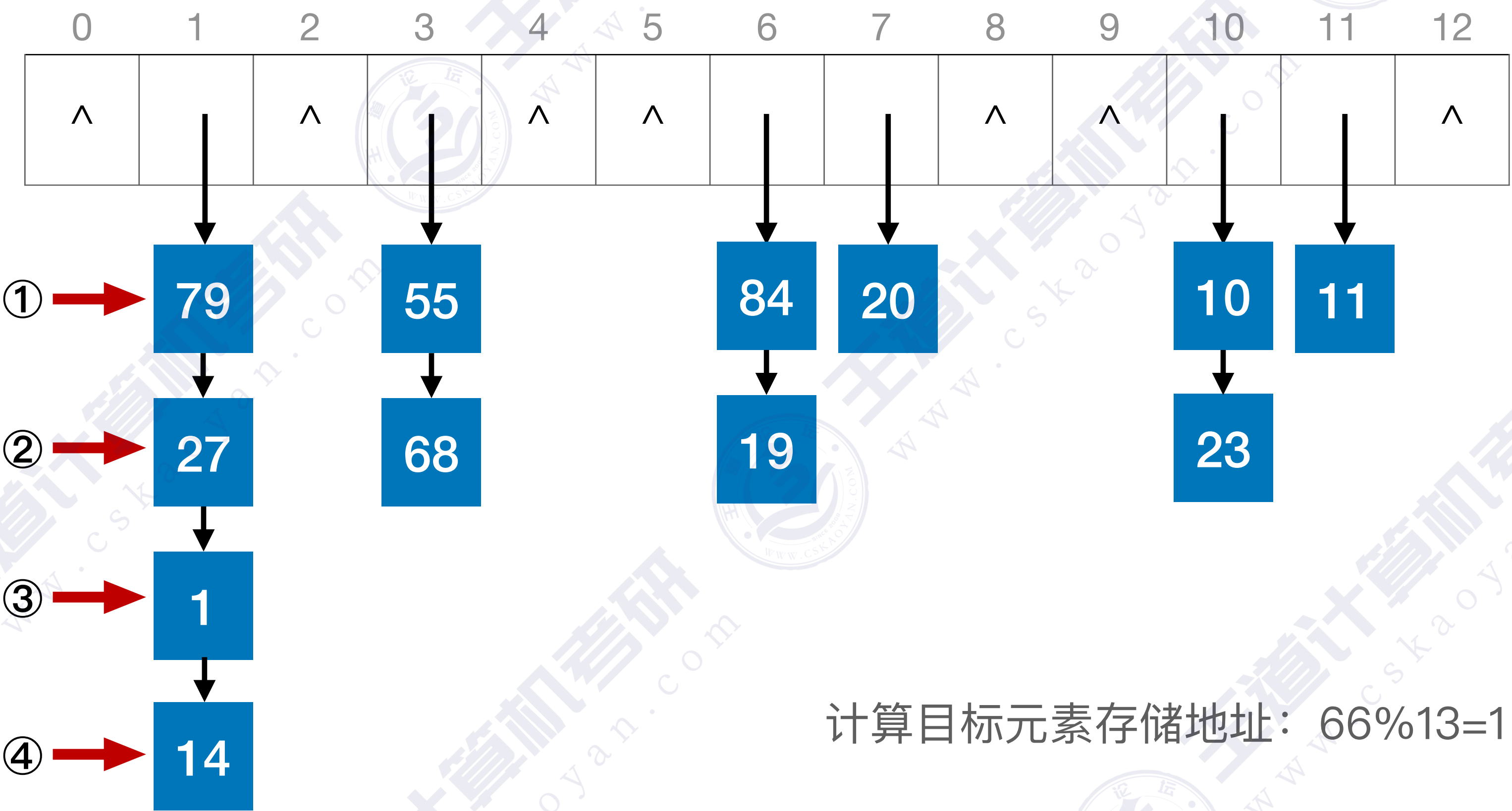
查找目标：20

计算目标元素存储地址： $20\%13=7$

20查找成功，查找长度=1

散列表的查找操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27，20，66，21，分析每次查找操作的过程和查找长度。



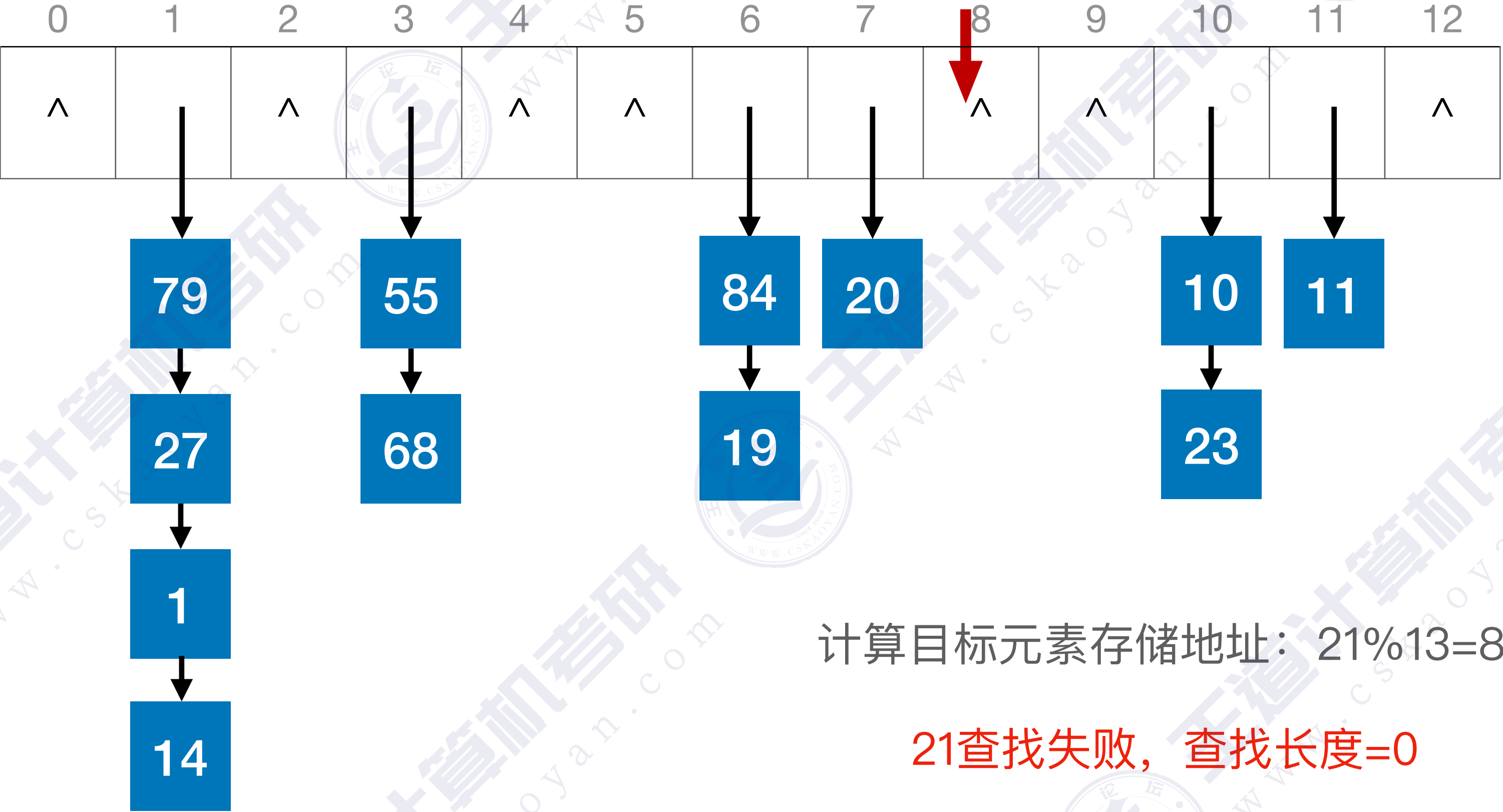
查找目标：66

计算目标元素存储地址： $66\%13=1$

66查找失败，查找长度=4

散列表的查找操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27，20，66，21，分析每次查找操作的过程和查找长度。



查找目标：21

计算目标元素存储地址： $21\%13=8$

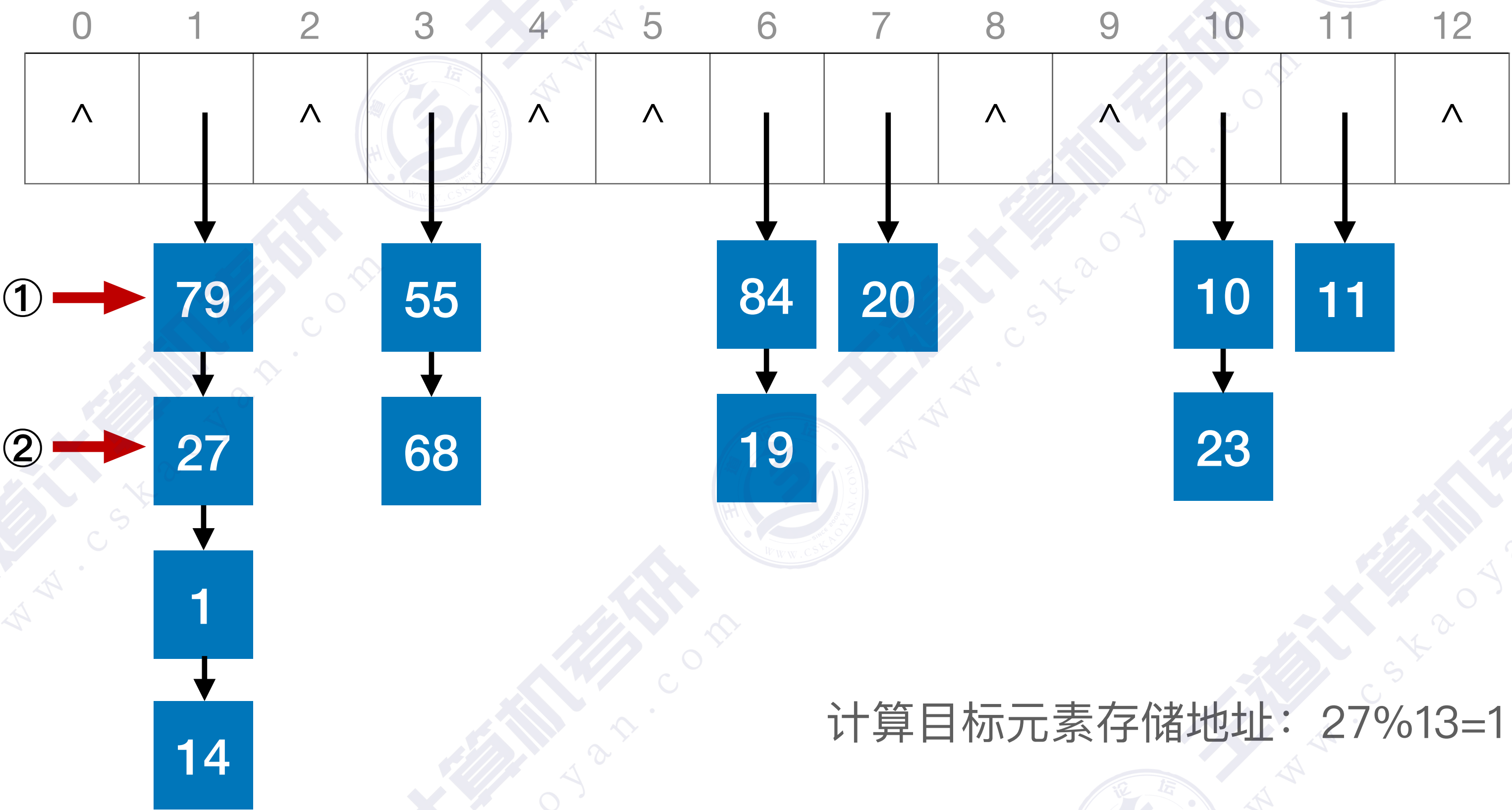
21查找失败，查找长度=0

注：有的教材会把“空指针的对比”也计入查找长度。但考试中默认只统计“关键字对比次数”。

查找长度——在查找运算中，需要对比关键字的次数称为查找长度

散列表的删除操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次删除元素 27，20，66。

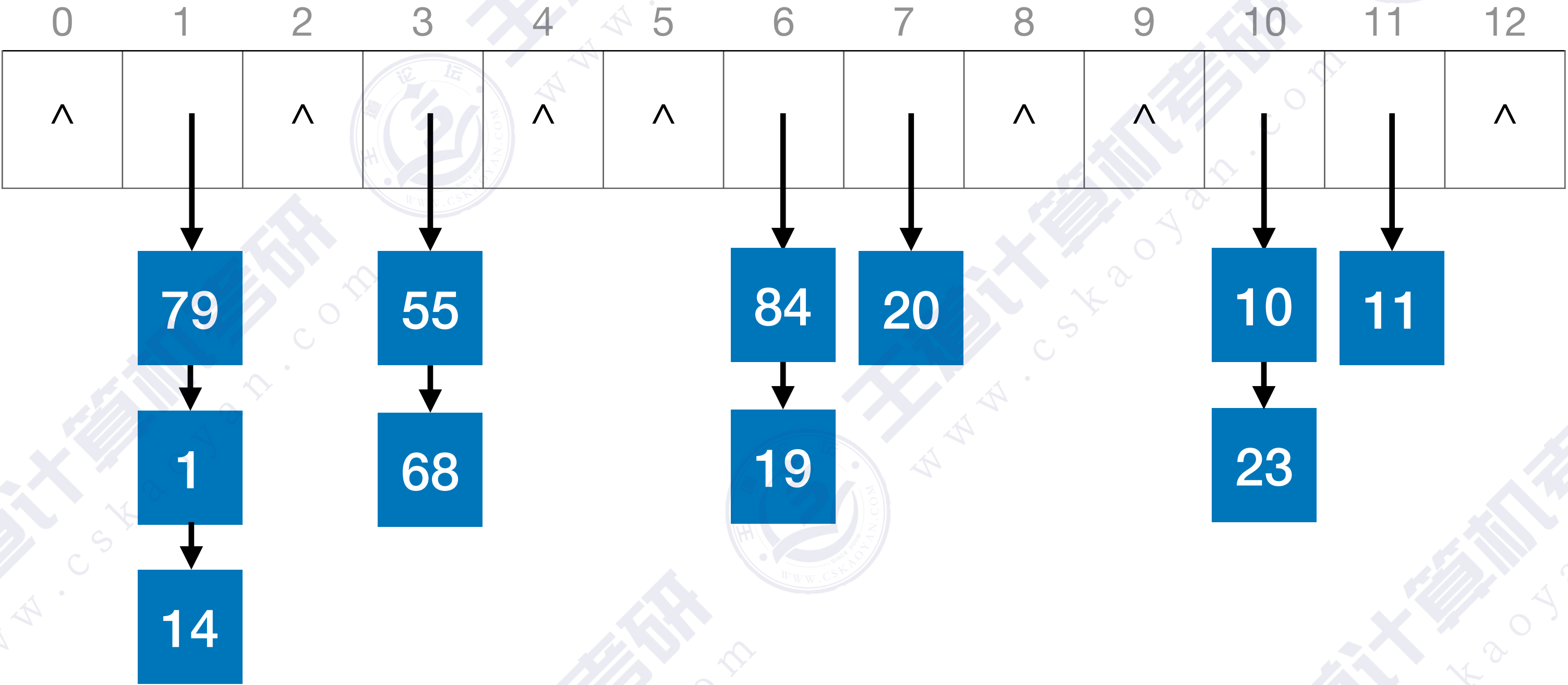


删除目标：27

计算目标元素存储地址： $27\%13=1$

散列表的删除操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次删除元素 27，20，66。



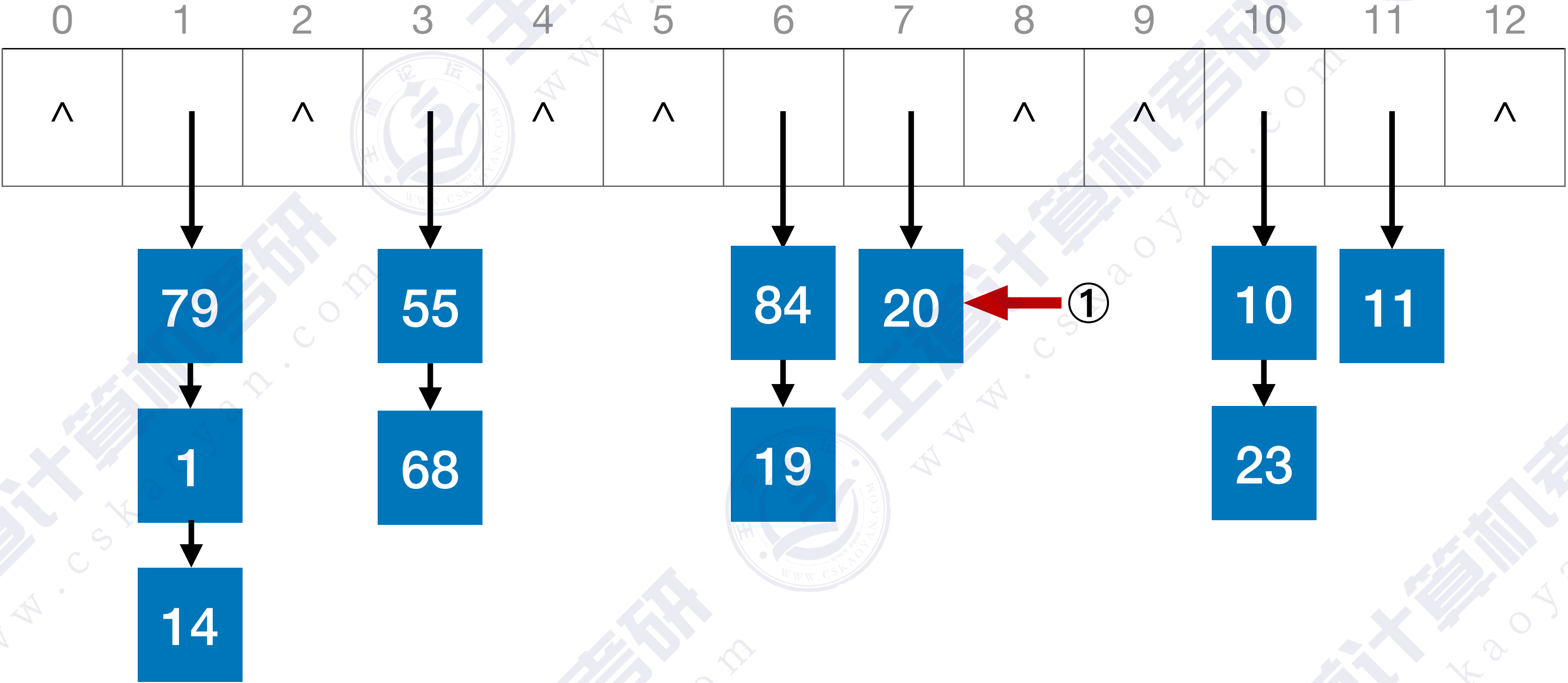
删除目标：27

计算目标元素存储地址： $27\%13=1$

27查找成功，删除成功

散列表的删除操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次删除元素 27，20，66。

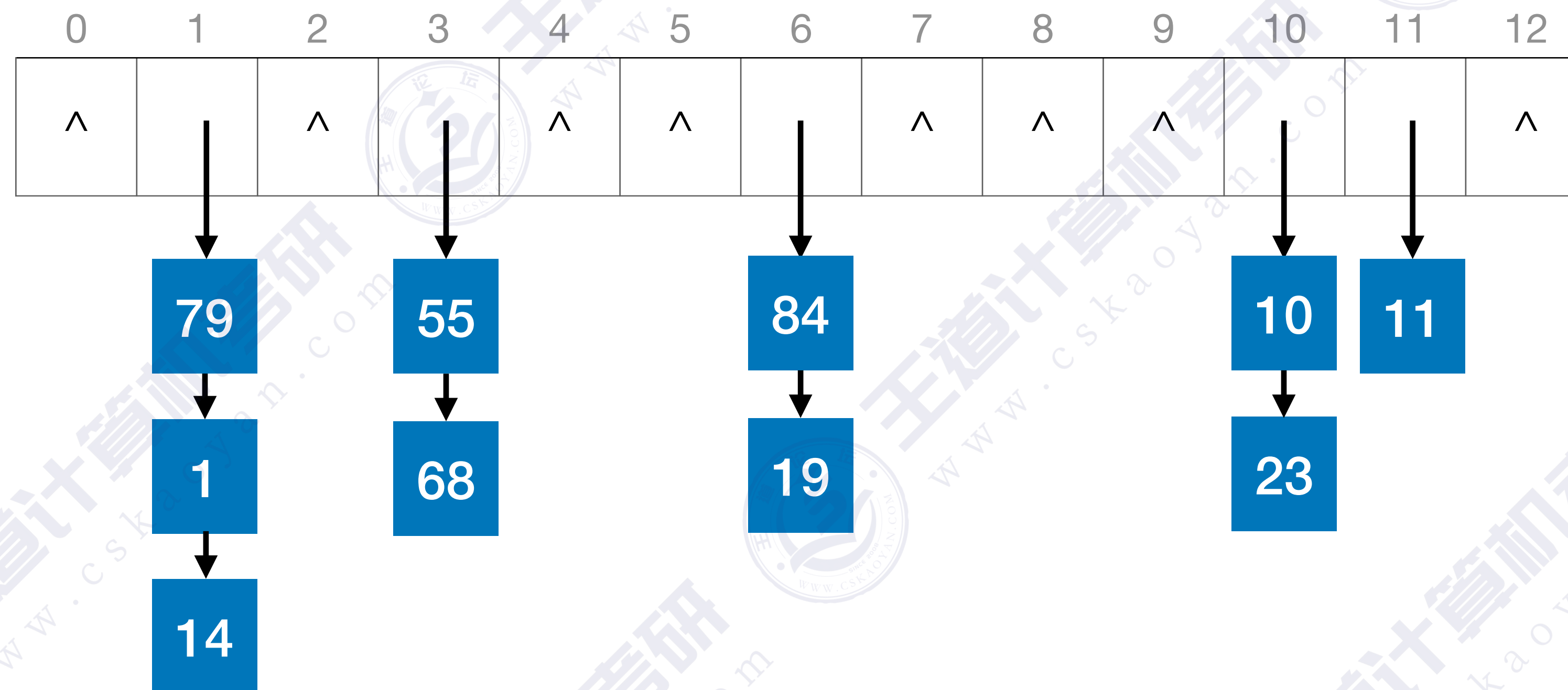


删除目标：20

计算目标元素存储地址： $20\%13=7$

散列表的删除操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。在下述散列表中依次删除元素 27, 20, 66。



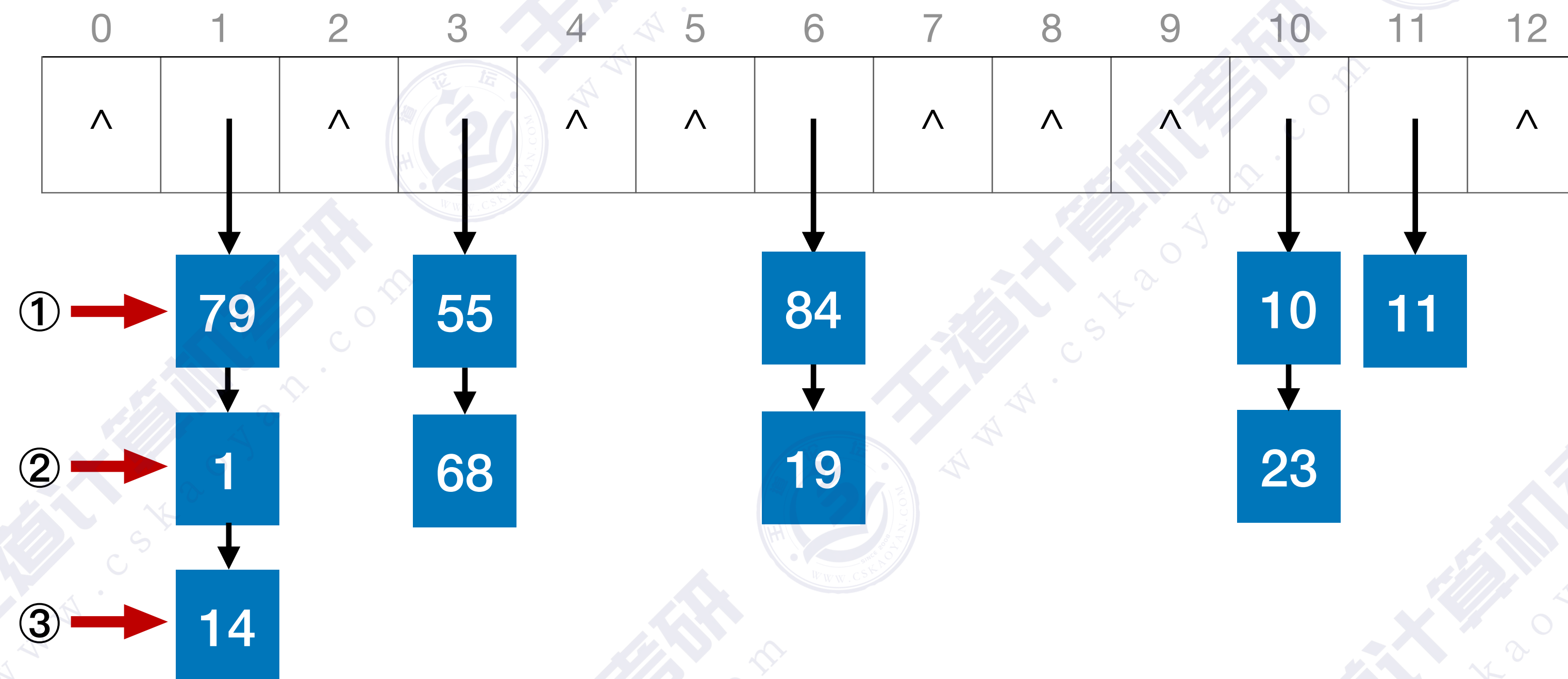
删除目标：20

计算目标元素存储地址： $20 \% 13 = 7$

20查找成功，删除成功

散列表的删除操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ ，用拉链法解决冲突。在下述散列表中依次删除元素 27, 20, 66。



删除目标：66

计算目标元素存储地址： $66 \% 13 = 1$

66查找失败，删除失败

知识回顾与重要考点

处理冲突的方法 ——拉链法

拉链法（又称链接法、链地址法）：把所有“同义词”存储在一个链表中

插入操作

Step 1: 根据散列函数计算新元素的散列地址

Step 2: 将新元素插入散列地址对应的链表（可用头插法或尾插法）

查找操作

Step 1: 根据散列函数计算目标元素的散列地址

Step 2: 顺序查找散列地址对应的链表，直到查找成功或查找失败

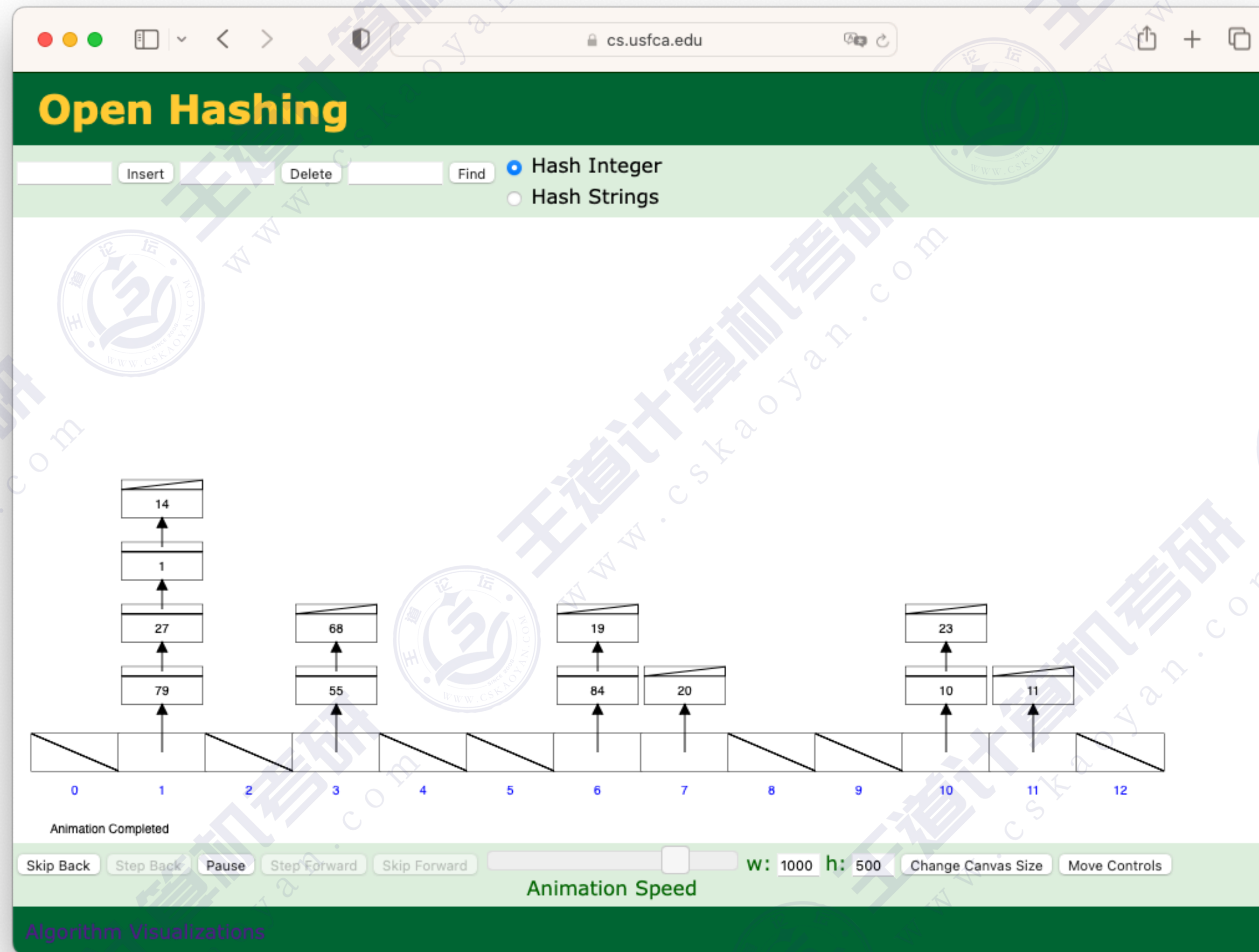
在分析查找长度时，通常只统计“关键字的对比次数”，而链表“空指针的对比次数”不计入查找长度

删除操作

Step 1: 根据散列函数计算目标元素的散列地址

Step 2: 顺序查找散列地址对应的链表，若查找成功，将目标元素从链表中删除

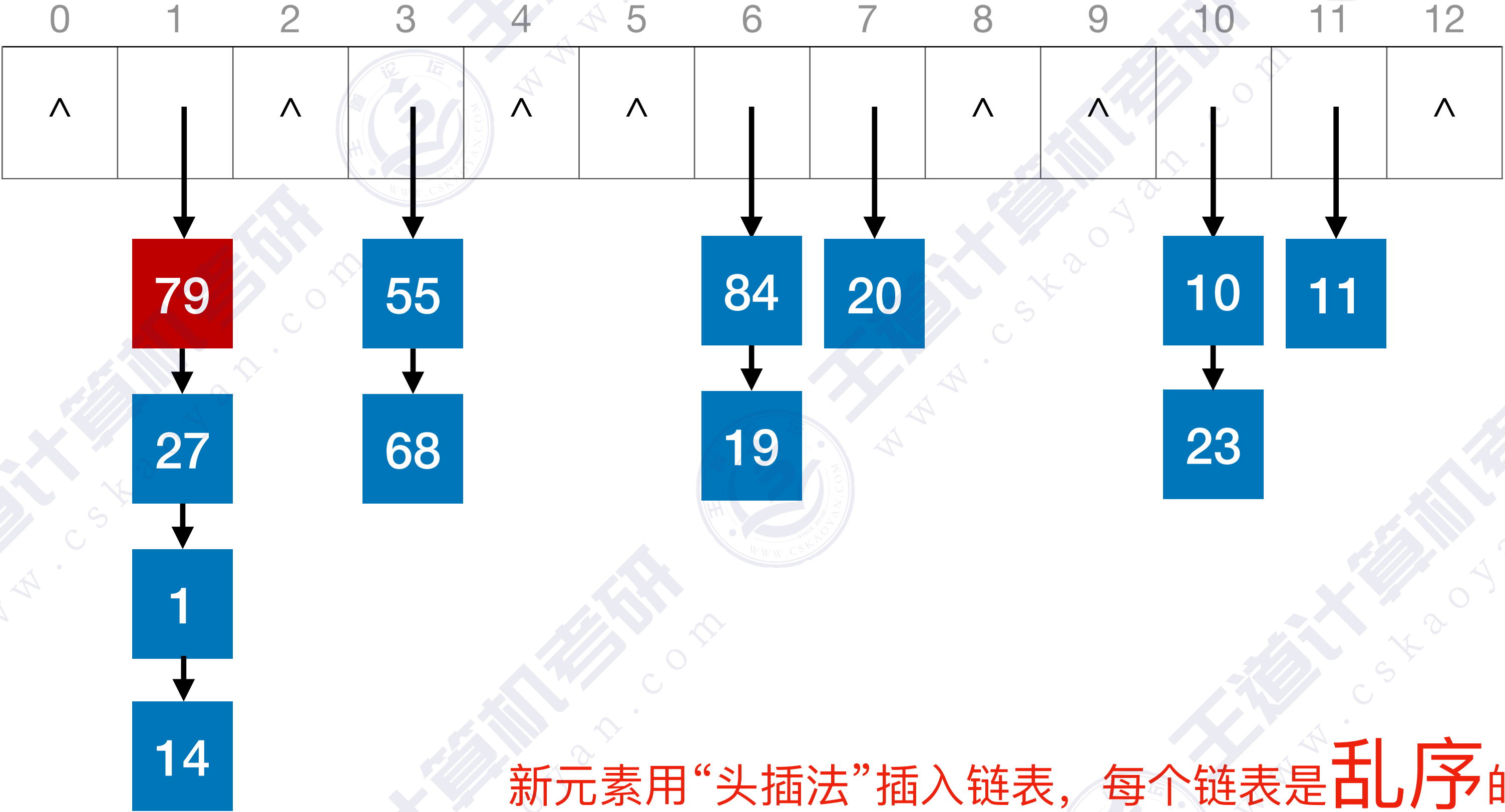
传送门



<https://www.cs.usfca.edu/~galles/visualization/OpenHash.html>

拓展：插入操作的小优化

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}



19%13=6

14%13=1

23%13=10

1%13=1

68%13=3

20%13=7

84%13=6

27%13=1

55%13=3

11%13=11

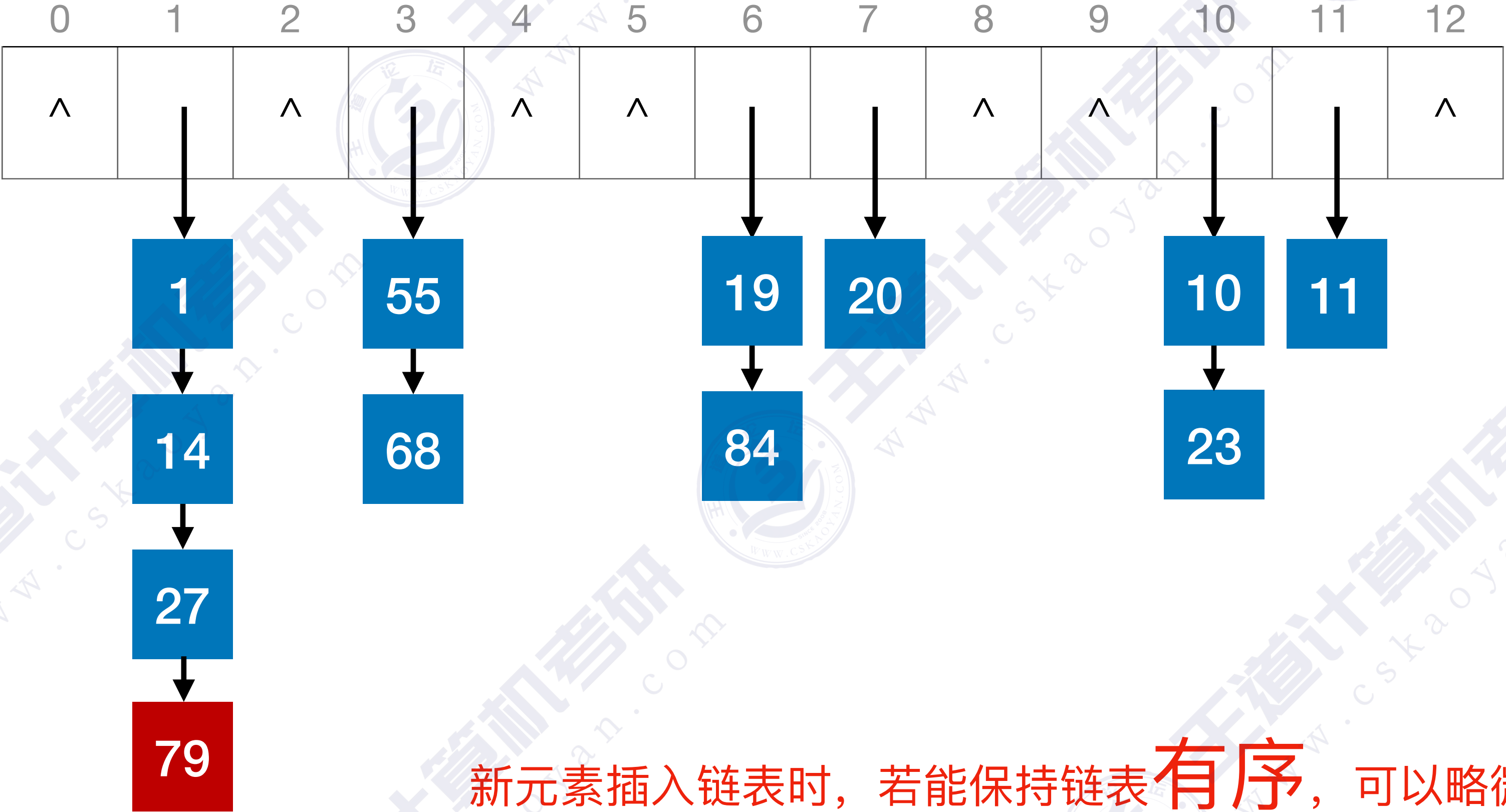
10%13=10

79%13=1

新元素用“头插法”插入链表，每个链表是乱序的。

拓展：插入操作的小优化

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}



19%13=6

14%13=1

23%13=10

1%13=1

68%13=3

20%13=7

84%13=6

27%13=1

55%13=3

11%13=11

10%13=10

79%13=1

新元素插入链表时，若能保持链表**有序**，可以略微提高“查找”效率。

本节内容

处理冲突的 方法

——开放定址法

知识总览

处理冲突的方法 ——开放定址法

开放定址法的原理

四种常用方法

★ 线性探测法

平方探测法

双散列法

伪随机序列法

如何处理“冲突”？——开放定址法

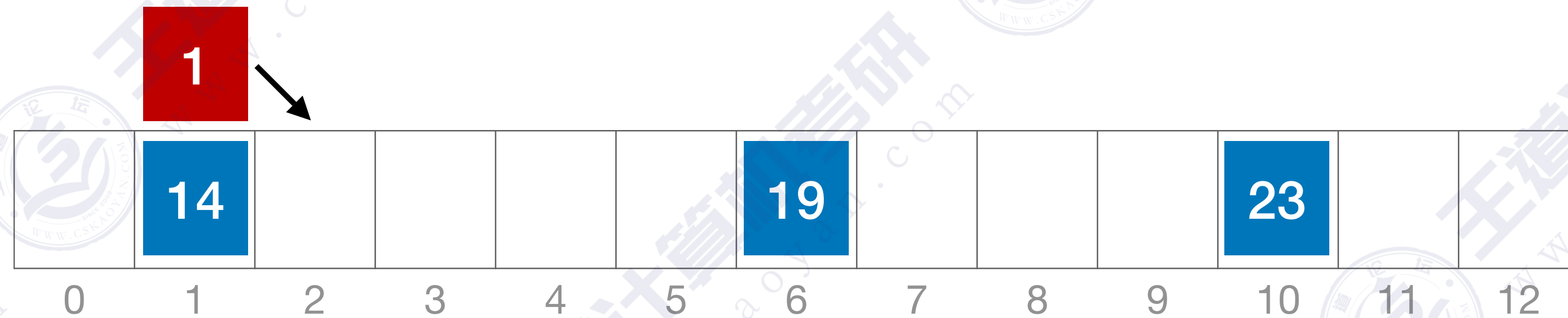
例：某散列表的长度为13，散列函数 $H(\text{key}) = \text{key} \% 13$ 。依次将数据元素 19、14、23、1 插入散列表：

$$19 \% 13 = 6$$

$$14 \% 13 = 1$$

$$23 \% 13 = 10$$

$$1 \% 13 = 1$$



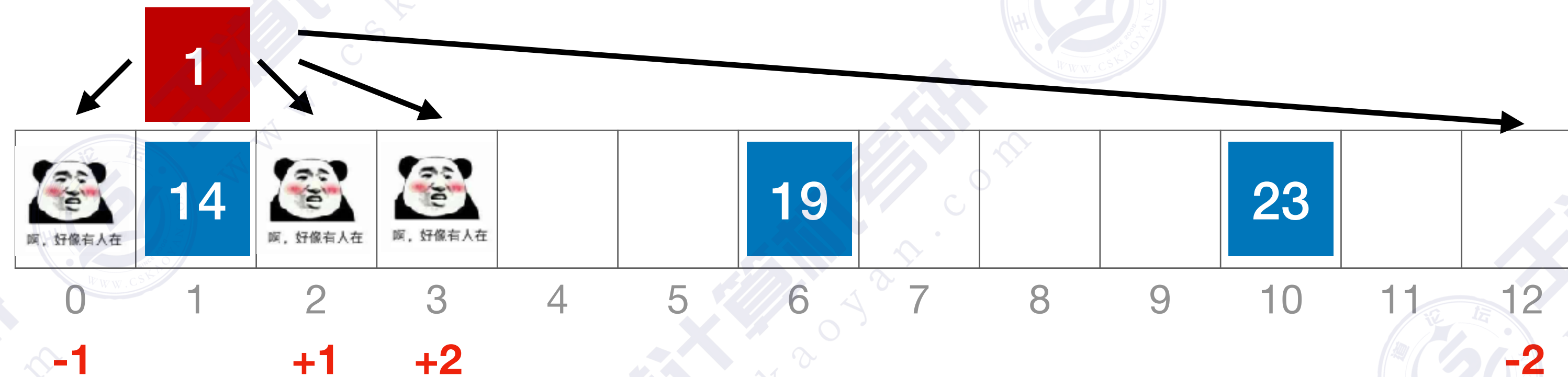
开放定址法：如果发生“冲突”，就给新元素找另一个空闲位置。

什么叫“开放定址”？—— 一个散列地址，既对同义词开放，也对非同义词开放。

开放定址法的基本原理

开放定址法：如果发生“冲突”，就给新元素找另一个空闲位置。

$19\%13=6$
 $14\%13=1$
 $23\%13=10$
 $1\%13=1$



待解决的问题：用什么规则确定“另一个空闲位置”？

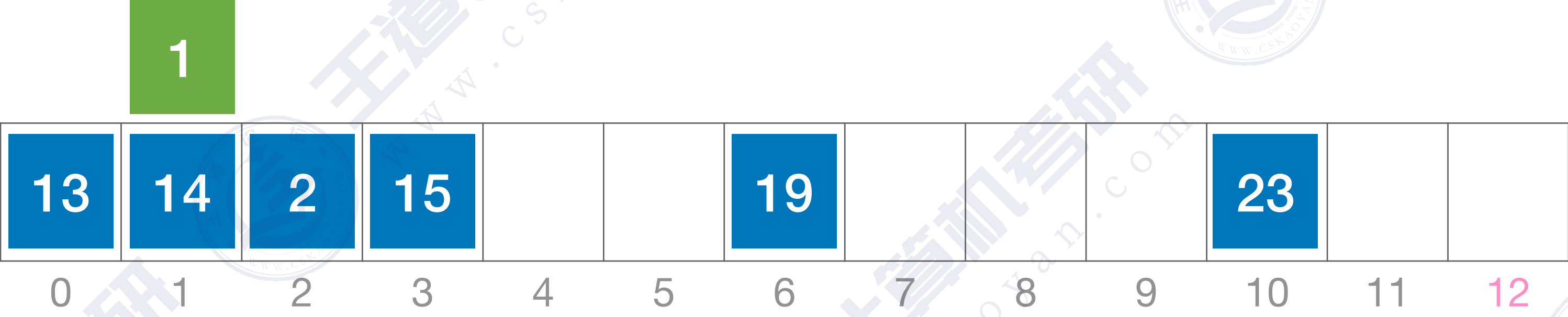
思路：需确定一个“探测的顺序”，从初始散列地址出发，去寻找下一个空闲位置。

eg: $d_0=0$, $d_1=1$, $d_2=-1$, $d_3=2$, $d_4=-2$,

注: d_i 表示第 i 次发生冲突时，下一个探测地址与初始散列地址的相对偏移量。

开放定址法的基本原理

根据散列函数 $H(key)$ ，求得初始散列地址。若发生冲突，如何找到“另一个空闲位置”？



发生第 i 次冲突时的散列地址

散列表表长

$$H_i = (H(key) + d_i) \% m$$

初始散列地址

偏移量

四种常用方法构造探测序列 d_i

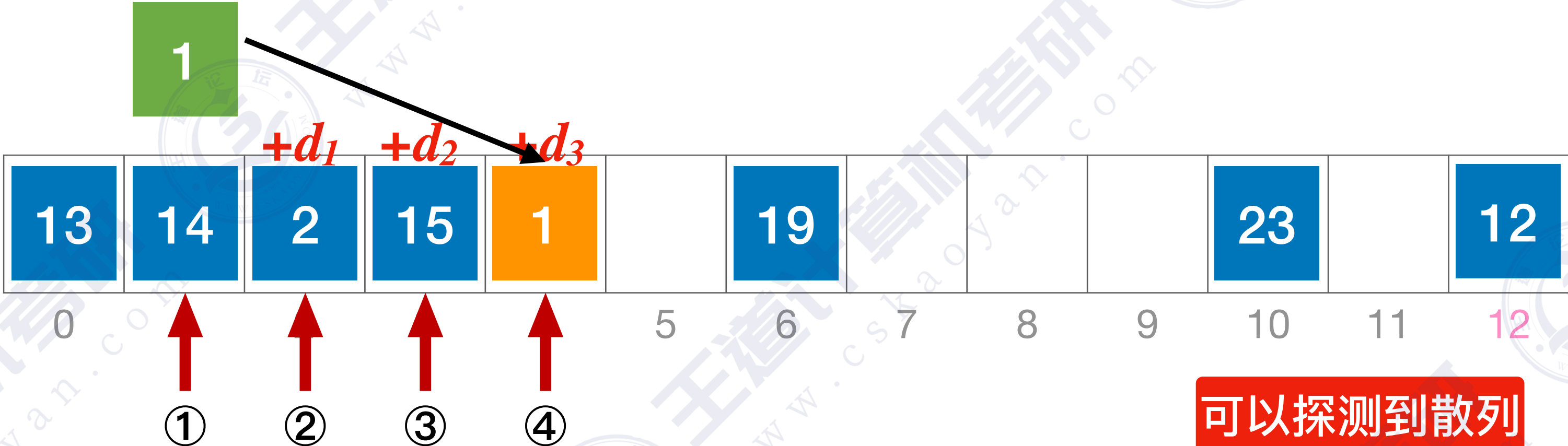
注： $0 \leq i \leq m-1$

探测序列/增量序列

- ★ 线性探测法 $d_i = 0, 1, 2, 3, \dots, m-1$
- 平方探测法 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 。其中 $k \leq m/2$
- 双散列法 $d_i = i \times hash_2(key)$ 。其中 $hash_2(key)$ 是另一散列函数
- 伪随机序列法 d_i 是一个伪随机序列，如 $d_i = 0, 5, 3, 11, \dots$

线性探测法（插入、查找操作）

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用线性探测法解决冲突。分析：插入元素1、查找元素1 的过程



可以探测到散列表的每个地址

注：查找操作原理类似，根据探测序列依次对比各存储单元内的关键字。
若探测到目标关键字，则查找成功。
若探测到空单元，则查找失败。

发生第 i 次冲突时的散列地址

散列表表长

线性探测法， $d_i = 0, 1, 2, 3, \dots, m-1$

初始散列地址 $H_0 = 1\%13 = 1$ ，发生冲突(第1次)

$H_1 = (1 + 1) \% 13 = 2$ ，发生冲突(第2次)

$H_2 = (1 + 2) \% 13 = 3$ ，发生冲突(第3次)

$H_3 = (1 + 3) \% 13 = 4$ ，未发生冲突，插入位置#4

初始散列地址

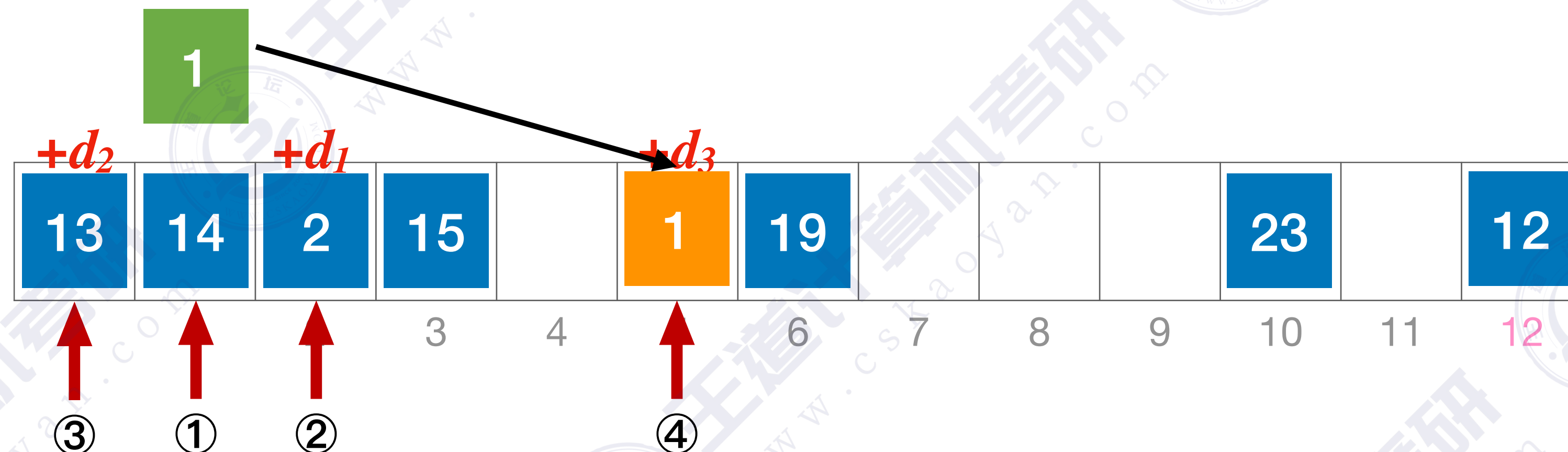
偏移量

$$H_i = (H(key) + d_i) \% m$$

平方探测法 (插入、查找操作)

又称“二次探测法”

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用平方探测法解决冲突。分析：插入元素1、查找元素1 的过程



发生第 i 次冲突时的散列地址

散列表
表长

$$H_i = (H(key) + d_i) \% m$$

初始散
列地址

偏移量

平方探测法， $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 。
其中 $k \leq m/2$

初始散列地址 $H_0 = 1\%13 = 1$ ，发生冲突(第1次)

$H_1 = (1 + 1) \% 13 = 2$ ，发生冲突(第2次)

$H_2 = (1 + -1) \% 13 = 0$ ，发生冲突(第3次)

$H_3 = (1 + 4) \% 13 = 5$ ，未发生冲突，插入位置#5

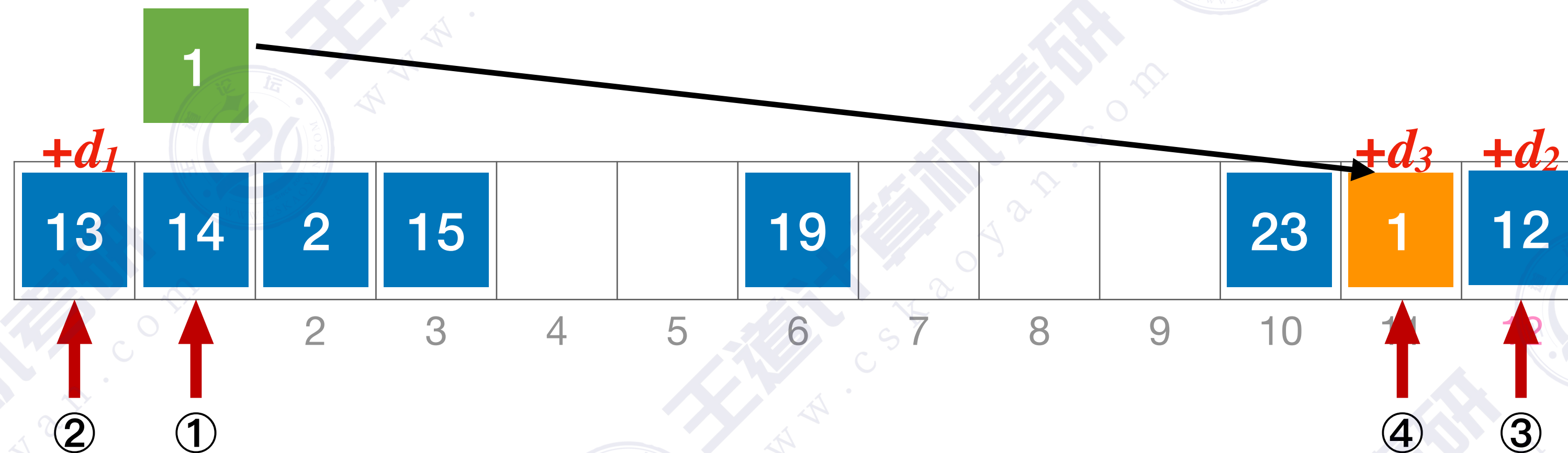
注：查找操作原理类似，根据探测序列依次对比各存储单元内的关键字。

若探测到目标关键字，则查找成功。

若探测到空单元，则查找失败。

双散列法（插入、查找操作）

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用双散列法解决冲突，假设 $hash_2(key)=13-(key\%13)$ 。分析：插入元素1、查找元素1 的过程



发生第 i 次冲突时的散列地址

$$H_i = (H(key) + d_i) \% m$$

初始散列地址

偏移量

散列表表长

双散列法， $d_i = i \times hash_2(key)$

初始散列地址 $H_0 = 1\%13 = 1$ ，发生冲突(第1次)

由于 $key=1$ ， $hash_2(key) = 13 - (key\%13) = 12$

$H_1 = (1 + 1 \times 12) \% 13 = 0$ ，发生冲突(第2次)

$H_2 = (1 + 2 \times 12) \% 13 = 12$ ，发生冲突(第3次)

$H_3 = (1 + 3 \times 12) \% 13 = 11$ ，未发生冲突，插入位置#11

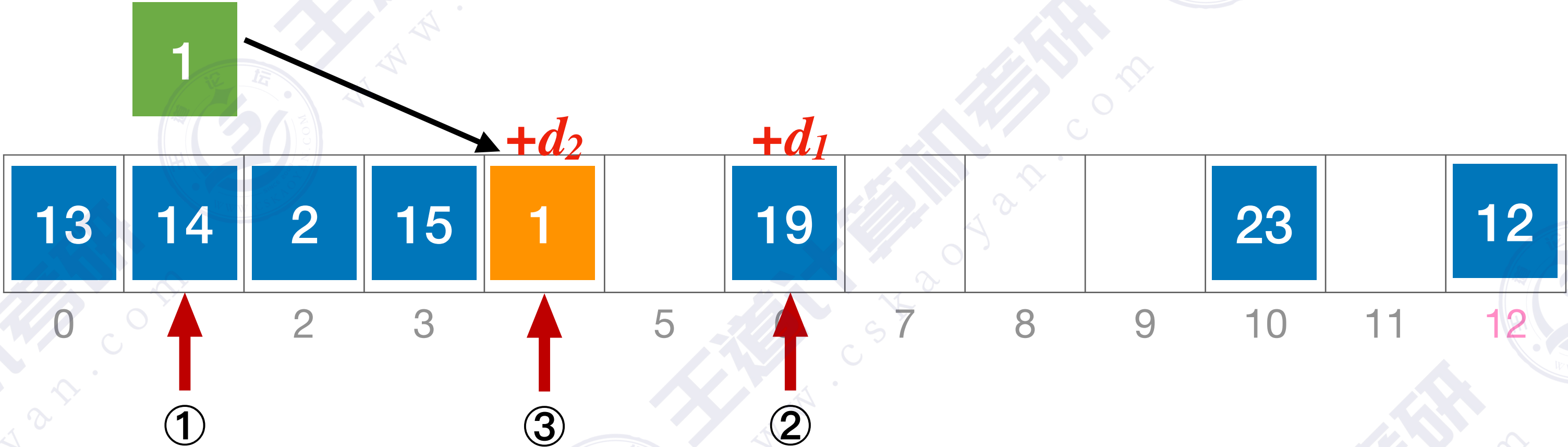
注：查找操作原理类似，根据探测序列依次对比各存储单元内的关键字。

若探测到目标关键字，则查找成功。

若探测到空单元，则查找失败。

伪随机序列法（插入、查找操作）

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用伪随机序列法解决冲突，假设伪随机序列 $d_i=0, 5, 3, 11, \dots$ ，其中 d_i 表示第 i 次发生冲突时的增量。分析：插入元素1、查找元素1 的过程



发生第 i 次冲突时的散列地址

散列表表长

d_i 是一个伪随机序列，由题目可知 $d_i=0, 5, 3, 11, \dots$

注：查找操作原理类似，根据探测序列依次对比各存储单元内的关键字。
若探测到目标关键字，则查找成功。
若探测到空单元，则查找失败。

$$H_i = (H(key) + d_i) \% m$$

初始散列地址

偏移量

初始散列地址 $H_0=1\%13=1$ ，发生冲突(第1次)
 $H_1 = (1 + 5) \% 13 = 6$ ，发生冲突(第2次)
 $H_2 = (1 + 3) \% 13 = 4$ ，未发生冲突，插入位置#4

如何删除一个元素？

注：题目一定会说明具体是采用哪种探测序列（线性探测法、平方探测法、双散列法、伪随机序列法）

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用开放定址法解决冲突。

13	14	2	15	1		19				23		12
0	1	2	3	4	5	6	7	8	9	10	11	12

如何删除一个元素：

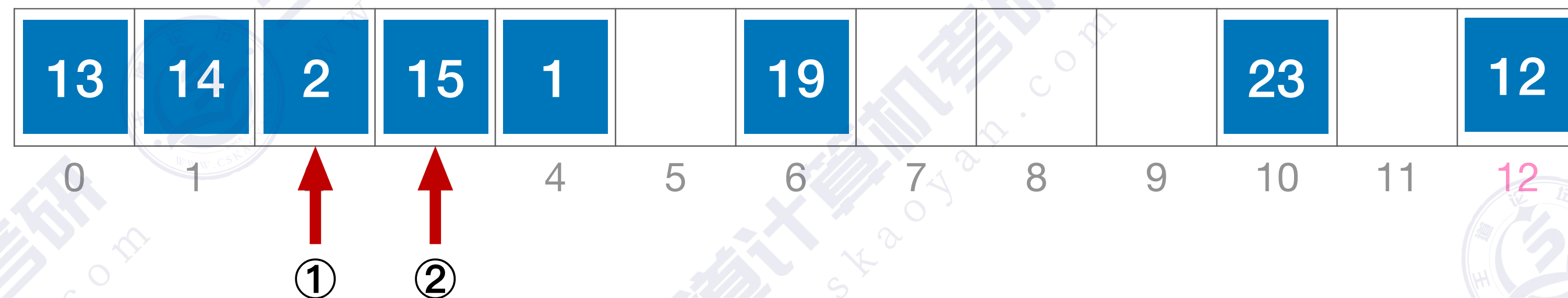
Step 1：先根据散列函数算出散列地址，并对比关键字是否匹配。若匹配，则“查找成功”

Step 2：若关键字不匹配，则根据“探测序列”对比下一个地址的关键字，直到“查找成功”或“查找失败”

Step 3：若“查找成功”，则删除找到的元素

特别注意：关于删除操作

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用线性探测法解决冲突。



错误示范：删除元素15

- 计算元素15 的初始散列地址= $15\%13=2$ 。对比位置#2，关键字不等于15；
- 根据线性探测法的探测序列，继续对比位置#3，关键字等于15；
- 删除元素15，清空位置#3

特别注意：关于删除操作

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用线性探测法解决冲突。



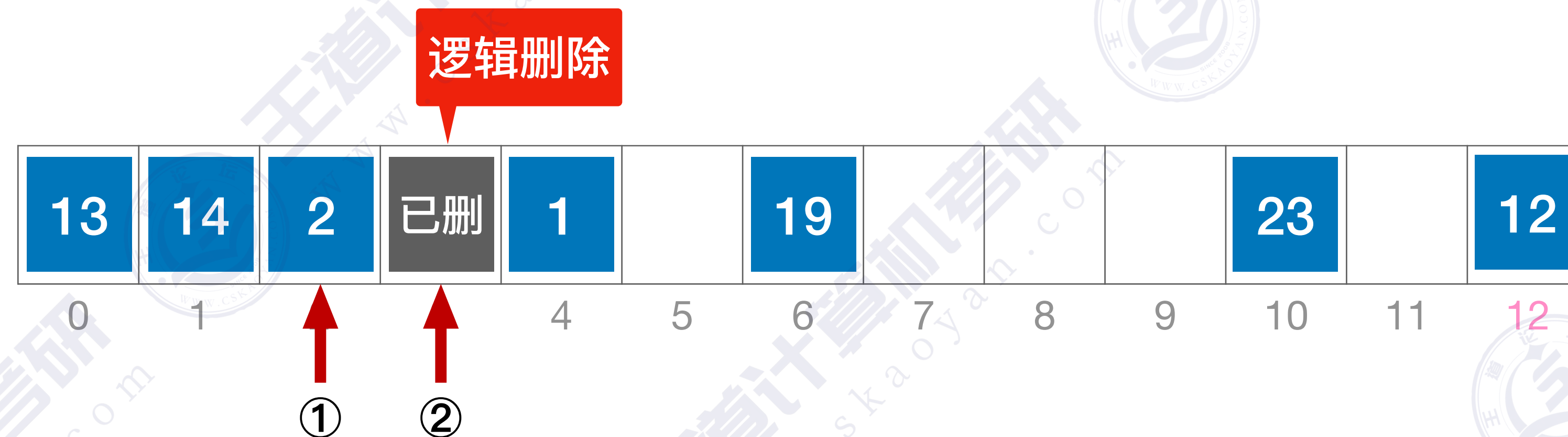
错误示范：查找元素1

- 计算元素1 的初始散列地址 $=1\%13=1$ 。对比位置#1，关键字不等于1；
- 根据线性探测法的探测序列，继续对比位置#2，关键字不等于1；
- 根据线性探测法的探测序列，继续对比位置#3，探测到空单元，查找失败。



特别注意：关于删除操作

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用线性探测法解决冲突。



正确示范：删除元素15

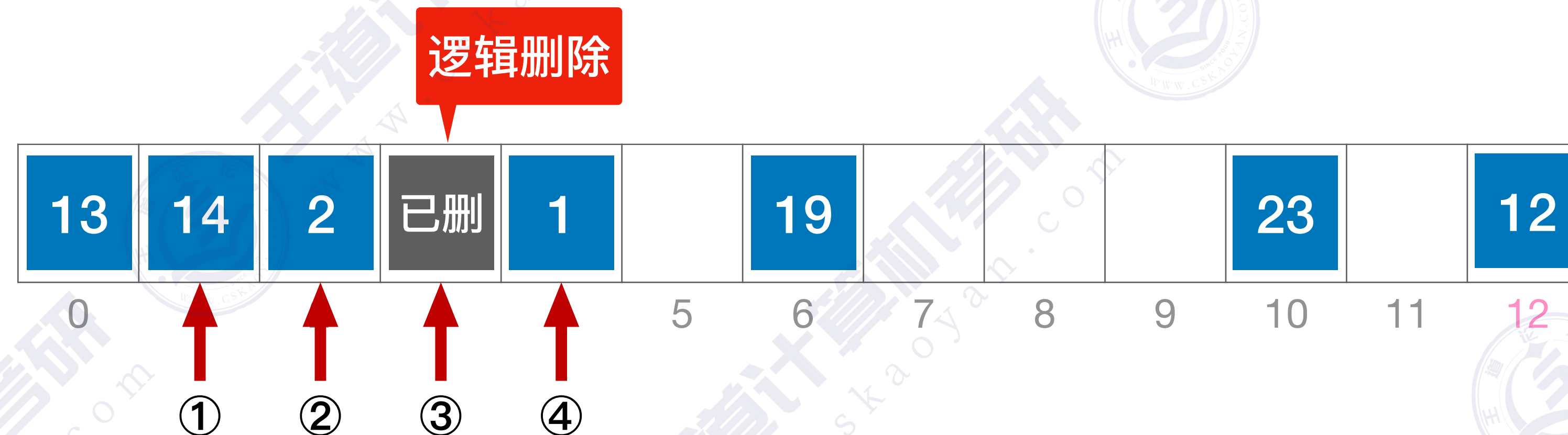
- 计算元素15 的初始散列地址= $15\%13=2$ 。对比位置#2，关键字不等于15；
- 根据线性探测法的探测序列，继续对比位置#3，关键字等于15；
- 逻辑删除元素15，将位置#3标记为“已删除”

注：无论线性探测法、平方探测法、双散列法、伪随机序列法原理都一样。删除元素时，只能逻辑删除

注意：采用“开放定址法”时，删除元素不能简单地将被删元素的空间置为空，否则将截断在它之后的探测路径，可以做一个“已删除”标记，进行逻辑删除。

特别注意：关于删除操作

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用线性探测法解决冲突。



正确示范：查找元素1

- 计算元素1 的初始散列地址= $1\%13=1$ 。对比位置#1，关键字不等于1；
- 根据线性探测法的探测序列，继续对比位置#2，关键字不等于1；
- 根据线性探测法的探测序列，继续对比位置#3，该位置原关键字已删，继续探测后一个位置；
- 根据线性探测法的探测序列，继续对比位置#4，关键字等于1，查找成功。

特别注意：关于删除操作

注意：采用“开放定址法”时，删除元素不能简单地将被删元素的空间置为空，否则将截断在它之后的探测路径，可以做一个“已删除”标记，进行逻辑删除。

整理前

已删	已删	已删	已删	1		19				23		12
0	1	2	3	4	5	6	7	8	9	10	11	12

注：新元素也可以插入到已被“逻辑删除”的地址

带来的问题：查找效率低下，散列表看起来很满，实则很空。

Tips：可以不定期整理散列表内的数据。

整理后

	1					19				23		12
0	1	2	3	4	5	6	7	8	9	10	11	12

脑子？空

知识回顾与重要考点

$$H_i = (H(\text{key}) + d_i) \% m$$

处理冲突的方法 ——开放定址法

基本原理

当初始散列地址发生冲突时，根据“探测序列” d_i 探测下一个地址

元素的操作

插入

根据散列函数算出初始散列地址，若发生冲突，就“探测”下一个地址，直到找到一个空闲地址，即可插入元素

查找

根据散列函数算出初始散列地址，对比关键字，若关键字不匹配，就“探测”下一个地址，直到关键字匹配(成功)或探测到一个空位置(失败)

删除

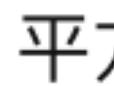
先按照“查找操作”的规则找到目标元素。若查找成功，就把目标元素“逻辑删除”。注意不能“物理删除”

四种常用的“探测序列”



线性探测法

$$d_i = 0, 1, 2, 3, \dots, m-1$$



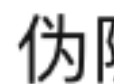
平方探测法

$$d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2。其中 k \leq m/2$$



双散列法

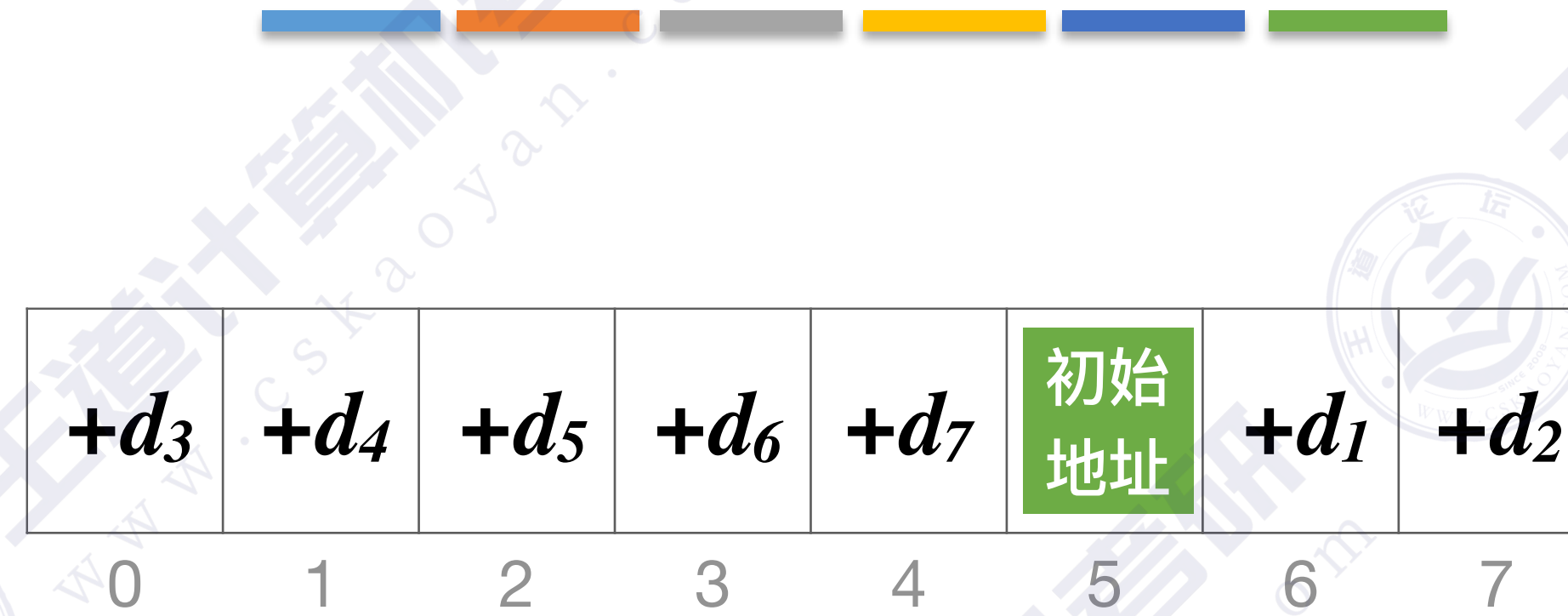
$$d_i = i \times \text{hash}_2(\text{key})。其中 \text{hash}_2(\text{key}) 是另一散列函数$$



伪随机序列法

$$d_i \text{ 是一个伪随机序列，如 } d_i = 0, 5, 3, 11, \dots$$

拓展：线性探测法的“探测覆盖率”



发生第 i 次冲突时的散列地址

散列表表长

线性探测法, $d_i = 0, 1, 2, 3, \dots, m-1$

$$H_i = (H(key) + d_i) \% m$$

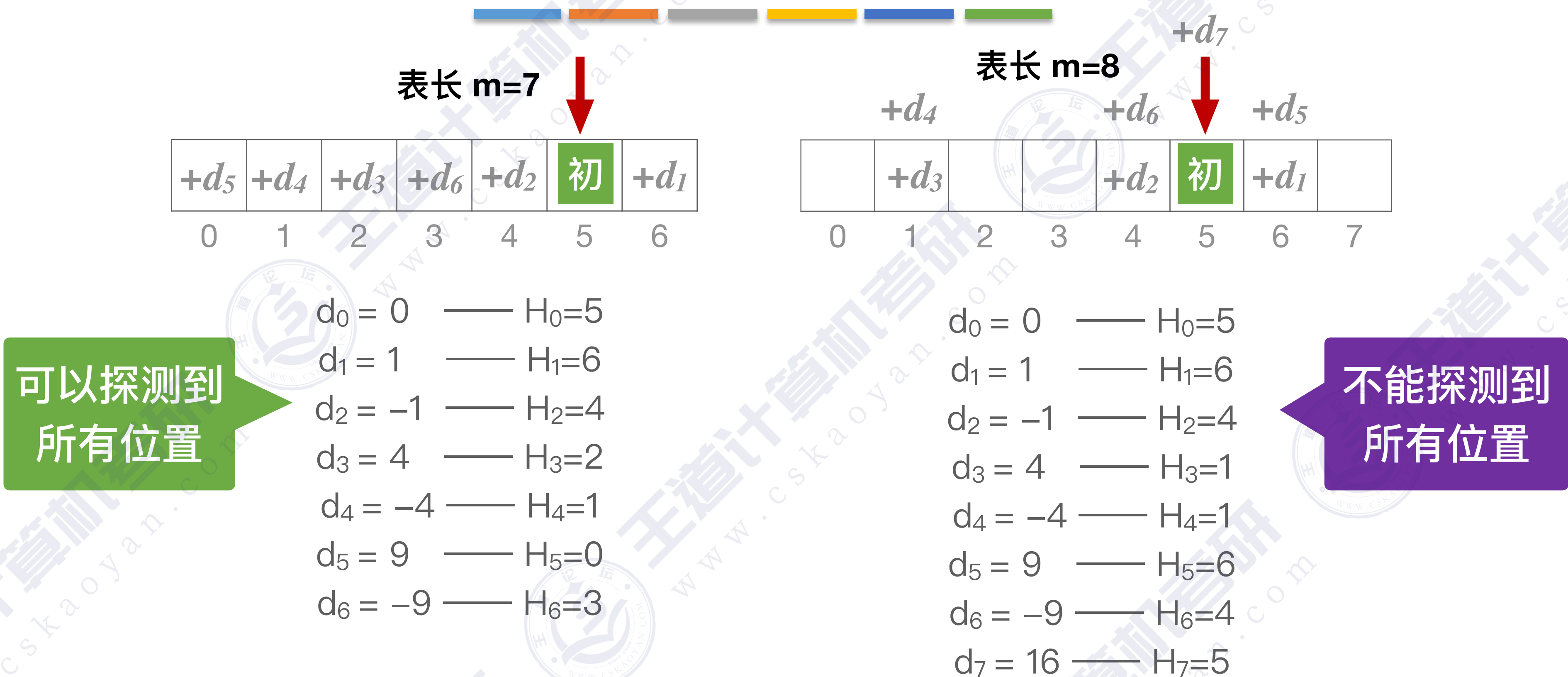
初始散列地址

偏移量

采用线性探测法, 一定可以探测到散列表的每个位置
只要散列表中有空闲位置, 就一定可以插入成功

理想情况下, 若散列表表长= m , 则最多发生 $m-1$ 次冲突即可“探测”完整个散列表。

拓展：平方探测法的“探测覆盖率”

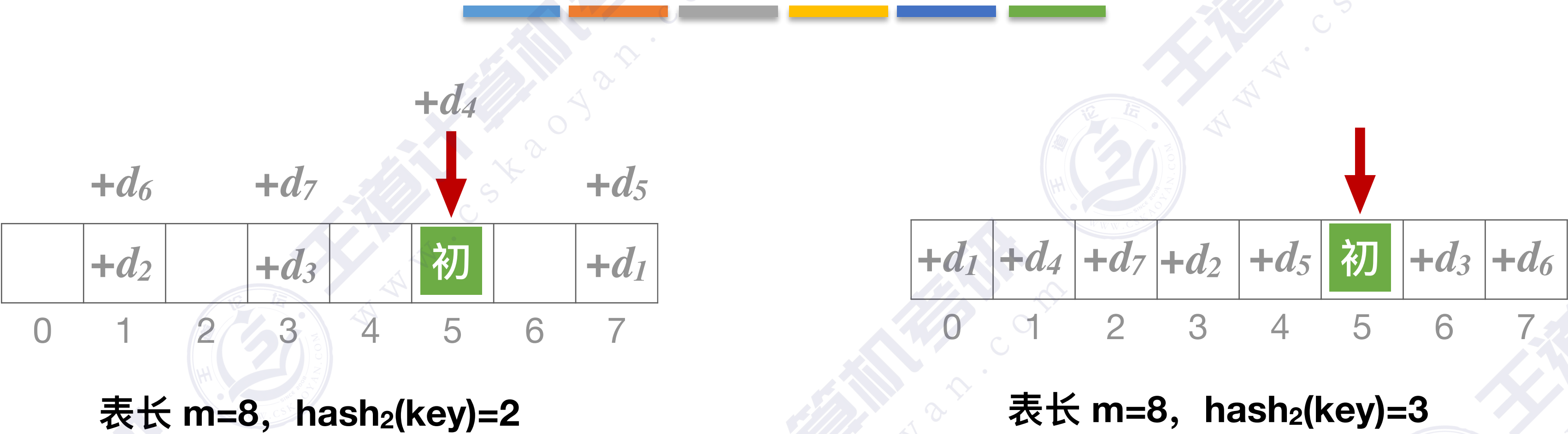


平方探测法， $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 。其中 $k \leq m/2$ ， $i \leq m-1$

采用平方探测法，至少可以探测到散列表中一半的位置
这意味着，即便散列表中有空闲位置，也未必能插入成功

若散列表长度 m 是一个可以表示成 $4j + 3$ 的素数(如 7、11、19)，平方探测法就能探测到所有位置

拓展：双散列法的“探测覆盖率”



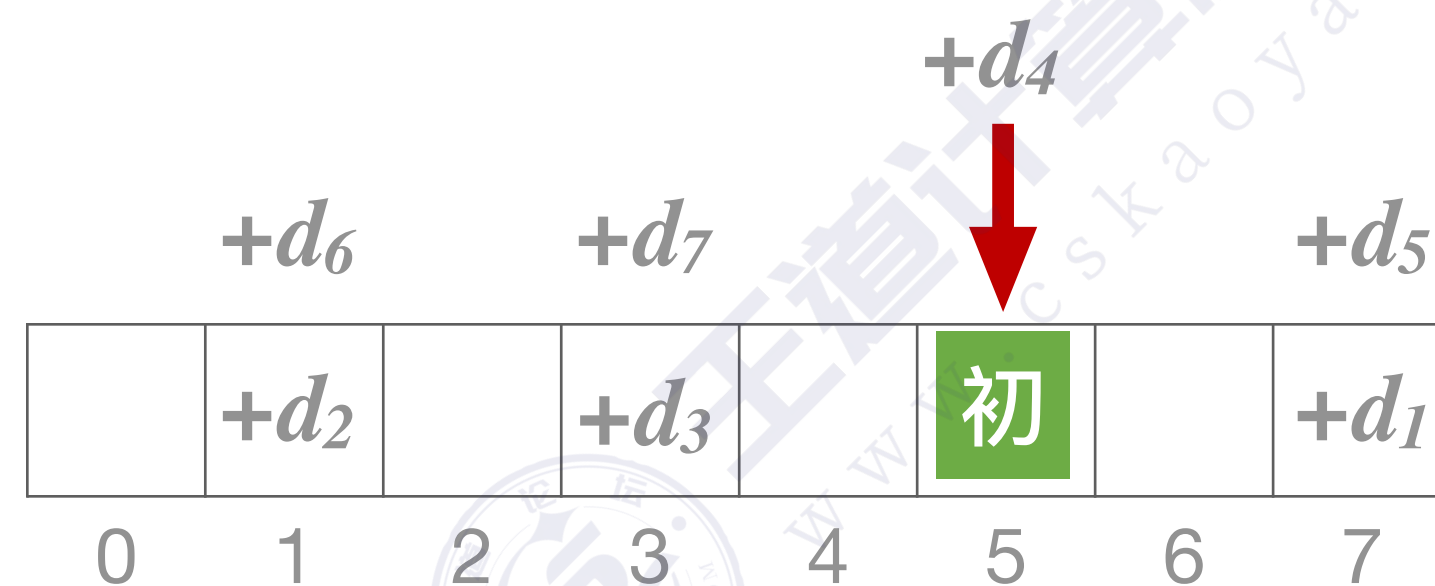
双散列法， $d_i = i \times \text{hash}_2(\text{key})$

双散列法未必能探测到散列表的所有位置。
双散列法的探测覆盖率取决于第二个散列函数 $\text{hash}_2(\text{key})$ 设计的是否合理。
若 $\text{hash}_2(\text{key})$ 计算得到的值与散列表表长 m 互质，就能保证双散列法可以探测所有单元

双散列法常用套路：令表长 m 本身就是质数， $\text{hash}_2(\text{key}) = m - (\text{key} \% m)$

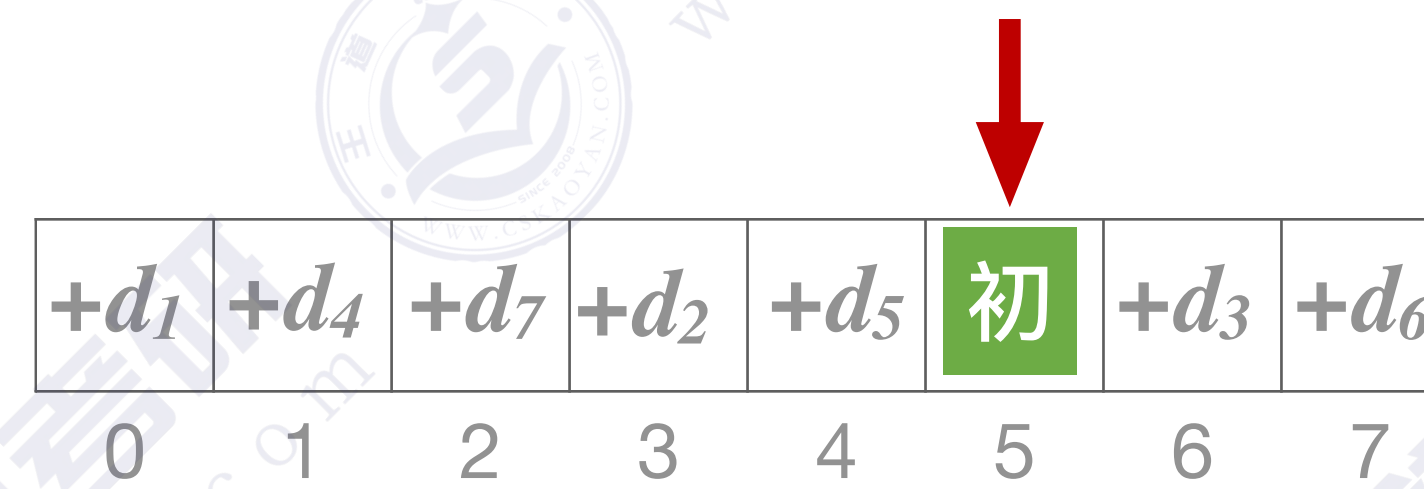
无论 key 值是多少，
 $\text{hash}_2(\text{key})$ 和 m 一定互质

拓展：伪随机序列法的“探测覆盖率”



表长 $m=8$

伪随机序列 $d_i=0,2,4,6,8,10,12,14$



表长 $m=8$

伪随机序列 $d_i=0,-5,-2,1,-4,-1,2,-3$

伪随机序列法： d_i 是一个伪随机序列，由程序员人为设计

采用伪随机序列法，是否能探测到散列表中全部位置，取决于伪随机序列的设计是否合理

拓展：四种探测序列的“探测覆盖率”



四种增量序列的“覆盖率”

- 线性探测法 —— 经过 $m-1$ 次冲突，一定能探测到散列表的所有单元
- 平方探测法 —— 一般来说，探测序列至少能覆盖到散列表的一半单元。若能保证表长 m 是一个可以表示成 $4X+3$ 的素数，则可以探测到散列表的所有单元
- 双散列法 —— 若能保证 $\text{hash2}(\text{key})$ 的值和表长 m 互质，则经过 $m-1$ 次冲突，一定能探测到散列表的所有单元
- 伪随机序列法 —— 只要伪随机序列设计合理，就能探测到全部单元

本节内容

散列查找的 性能分析

以线性探测法为例

知识总览

散列查找的性能分析

如何计算散列表的ASL? (以线性探测法为例)

影响散列查找性能的因素

装填因子

聚集 (堆积) 现象

散列查找的性能分析

例：设散列表HT长度为16，初始为空。散列函数 $H(key)=key\%13$ ，处理冲突采用线性探测法。依次插入关键字序列 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

- (1) 请画出所构造的散列表
- (2) 请计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。

	14	1	68	27	55	19	20	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

计算每个关键字的初始散列地址，若发生冲突，采用线性探测法解决冲突：

$$H(19) = 19\%13 = 6$$

$$H(14) = 14\%13 = 1$$

$$H(23) = 23\%13 = 10$$

$$H(1) = 1\%13 = 1 \rightarrow 2$$

$$H(68) = 68\%13 = 3$$

$$H(20) = 20\%13 = 7$$

$$H(84) = 84\%13 = 6 \rightarrow 7 \rightarrow 8$$

$$H(27) = 27\%13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$$

$$H(55) = 55\%13 = 3 \rightarrow 4 \rightarrow 5$$

$$H(11) = 11\%13 = 11$$

$$H(10) = 10\%13 = 10 \rightarrow 11 \rightarrow 12$$

$$H(79) = 79\%13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$$

$$ASL_{成功} = \frac{1 + 1 + 1 + 2 + 1 + 1 + 3 + 4 + 3 + 1 + 3 + 9}{12} = 2.5$$

计算“查找成功”的ASL：假设查找每一个已存在关键字的概率相同

散列查找的性能分析

例：设散列表HT长度为16，初始为空。散列函数 $H(key)=key\%13$ ，处理冲突采用线性探测法。依次插入关键字序列 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

(1) 请画出所构造的散列表

(2) 请计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。

Tips: 任何一个关键字的初始散列地址都不可能落在13~15

	14	1	68	27	55	19	20	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

啊~对对对
太^辣



$$ASL_{\text{失败}} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{13} = 7$$

计算“查找失败”的ASL：假设出现每一个初始散列地址的概率相同



错错错，是我的错

$$ASL_{\text{失败}} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 1 + 1}{16}$$

经典错误！注意：虽然散列表长度为16，但散列函数的取值范围仅有13种可能性

散列查找的性能分析

例：设散列表HT长度为16，初始为空。散列函数 $H(key)=key\%13$ ，处理冲突采用线性探测法。依次插入关键字序列 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

(3) 若删除元素20，请再计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。

	14	1	68	27	55	19	逻辑删除	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$$H(19) = 19\%13 = 6$$

$$H(14) = 14\%13 = 1$$

$$H(23) = 23\%13 = 10$$

$$H(1) = 1\%13 = 1 \rightarrow 2$$

$$H(68) = 68\%13 = 3$$

$$H(84) = 84\%13 = 6 \rightarrow 7 \rightarrow 8$$

$$H(27) = 27\%13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$$

$$H(55) = 55\%13 = 3 \rightarrow 4 \rightarrow 5$$

$$H(11) = 11\%13 = 11$$

$$H(10) = 10\%13 = 10 \rightarrow 11 \rightarrow 12$$

$$H(79) = 79\%13 = 1 \rightarrow 2 \rightarrow \dots \rightarrow 9$$

$$ASL_{成功} = \frac{1 + 1 + 1 + 2 + 1 + 3 + 4 + 3 + 1 + 3 + 9}{11} = 2.63$$

$$ASL_{失败} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{13} = 7$$



注意！注意！

注意易错点：采用开放定址法时，删除一个元素是逻辑删除，而非物理删除！这一点将影响查找长度的计算

装填因子

散列表的装填因子 $\alpha = \frac{\text{表中记录数 } n}{\text{散列表长度 } m}$

反映了一个散列表
“满”的程度

显然，装填因子越大，越容易发生冲突。从而导致插入、查找操作效率降低，ASL增大

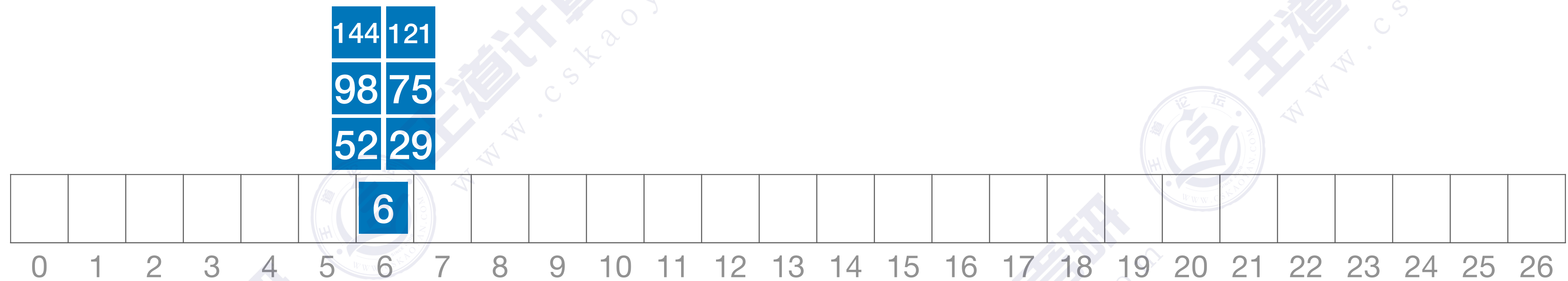
	14	1	68	27	55	19	20	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

例：如上所示，散列表HT长度为16，散列函数 $H(\text{key}) = \text{key} \% 13$ ，其中已插入12个元素，则装填因子 $\alpha = 12/16 = 0.75$

聚集（堆积）现象



例：散列表长度为27，散列函数 $H(\text{key}) = \text{key} \% 23$ ，采用线性探测法处理冲突



聚集（堆积）现象

例：散列表长度为27，散列函数 $H(\text{key}) = \text{key} \% 23$ ，采用线性探测法处理冲突

散列地址 #13，将被初始散列地址 6、7、8、9、10、11、12、13 的元素争夺

						6	29	52	75	98	121	144	?														
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	

聚集（堆积）现象：在处理冲突的过程中，几个初始散列地址不同的元素争夺同一个后继散列地址的现象称作“聚集”（或称作“堆积”）

线性探测法在发生冲突时，总是往后探测相邻的后一个单元，很容易造成同义词、非同义词的“聚集（堆积）现象”，从而影响查找效率，导致ASL提升

回顾开放定址法的几种常用“探测序列”

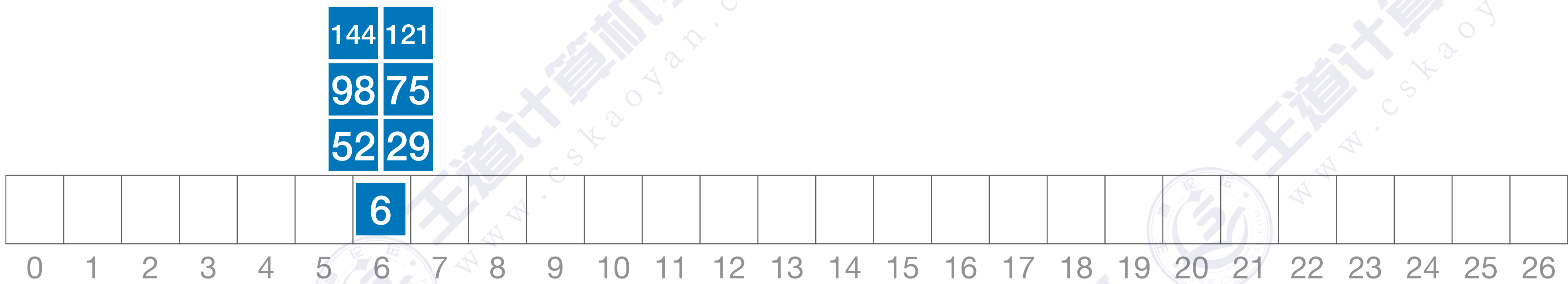


开放定址法： $H_i = (H(\text{key}) + d_i) \% m$ ，其中， H_i 表示第 i 次发生冲突时的散列地址； $i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列；

- 四种常用的“探测序列”
- ★ 线性探测法 $d_i = 0, 1, 2, 3, \dots, m-1$
 - 平方探测法 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 。其中 $k \leq m/2$
 - 双散列法 $d_i = i \times \text{hash}_2(\text{key})$ 。其中 $\text{hash}_2(\text{key})$ 是另一散列函数
 - 伪随机序列法 d_i 是一个伪随机序列，如 $d_i = 0, 5, 3, 11, \dots$

使用“平方探测法”减少聚集现象

例：散列表长度为27，散列函数 $H(key)=key\%23$ ，采用平方探测法（二次探测法）处理冲突



平方探测法（又称二次探测法）： $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ ，其中 $k \leq m/2$

$d_0 = 0, d_1 = 1, d_2 = -1, d_3 = 4, d_4 = -4, d_5 = 9, d_6 = -9 \dots$

平方探测法（又称二次探测法）： $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$, 其中 $k \leq m/2$

结论：采用线性探测法处理冲突更容易导致“聚集（堆积）现象”，用其他方法处理冲突可以将元素“打散”，从而减少“聚集（堆积）现象”

知识回顾与重要考点

散列查找的性能分析

如何计算散列表的ASL

$$ASL_{成功} = \frac{1}{n} \sum_{i=1}^n C_i$$

其中， n 为散列表中已存在的元素个数， C_i 为成功查找第 i 个元素所需的比较次数

$$ASL_{失败} = \frac{1}{r} \sum_{i=1}^r C_i$$

其中， r 为散列函数取值的个数， C_i 为散列函数取值为 i 时查找失败的比较次数

特别注意：散列函数取值的个数、散列表长度可能不相同

装填因子

$$\alpha = \frac{\text{表中记录数 } n}{\text{散列表长度 } m}$$

——反映了一个散列表“满”的程度

装填因子越大，越容易发生冲突。从而导致插入、查找效率降低，ASL增大

聚集（堆积）现象

在处理冲突的过程中，几个初始散列地址不同的元素争夺同一个后继散列地址的现象称作“聚集”（或称作“堆积”）

聚集（堆积）现象会影响查找效率，导致ASL提升

采用线性探测法处理冲突更容易导致“聚集（堆积）现象”，用其他方法处理冲突可以将元素“打散”，从而减少“聚集（堆积）现象”